NOURISHING LIVES- an attempt to touch each life

OSMANIA UNIVERSITY

HYDERABAD

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE

SUBMITTED BY

1. **Meher Nigeban** **(123822467017)**
2. **Ruqsar Begum** **(123822467023)**
3. **Shehla Ahmed** **(123822467027)**
4. **T.Sanjana** **(123822467031)**

DEPARTMENT OF COMPUTER SCIENCE

VILLA MARIE DEGREE COLLEGE FOR WOMEN

(Affiliated to Osmania University) Somajiguda,

Hyderabad – 500082

2022-2025

Acknowledgement

We would like to place on record our deep sense of gratitude to Mrs. S. Gayathri, HOD Department of Computer Science, Villa Marie Degree College for Women, Hyderabad, for her stimulating guidance, continuous encouragement and supervision through the course of the present work.

We also wish to extend our thanks to the Department Lecturers and other colleagues for attending our seminars and for the insightful comments and constructive suggestions to improve the quality of this project work.

We are extremely thankful to Mrs. Revati Devi Mathur, Principal of Villa Marie Degree College for Women, Hyderabad, for providing us her valuable suggestions.

We are extremely thankful to Ms. Chinnamma, Secretary of Villa Marie Degree College for Women and Dr. Philomena, Founder and Director of Villa Marie Degree College for Women, Hyderabad, for providing us infrastructural facilities to work in, without which this work would not have been possible.

Signature of the Students

1. Meher Nigeban (123822467017)
2. Ruqsar Begum (123822467023)
3. Shehla Ahmed (123822467027)
4. T.Sanjana (123822467031)

Declaration

We hereby declare that this project report entitled "Nourishing Lives-an attempt to touch each life" submitted by us to the Department of Computer Science, Osmania University, Hyderabad, is a bonafide work undertaken by us and it is not submitted to any other university or institution for the award of any degree/diploma/certificate or published any time before.

Signature of the Students

1. Meher Nigeban (123822467017)

2. Ruqsar Begum (123822467023)

3. Shehla Ahmed (123822467027)

4. T.Sanjana (123822467031)

Table of Content

Sno.	Title	Page No.
1	Abstract	5-6
2	System Analysis	7
2.1	Study of Existing System	8
2.2	Literature Survey	9-12
2.3	Problem Definition	13
2.4	Proposed System	14
2.5	Objective	15
3	Requirement Specification	16-17
4	Design	18-19
5	Coding	20-82
6	Partial Implementation	83-86
7	Conclusion	87-88
8	Bibliography	89-90

ABSTRACT

Abstract

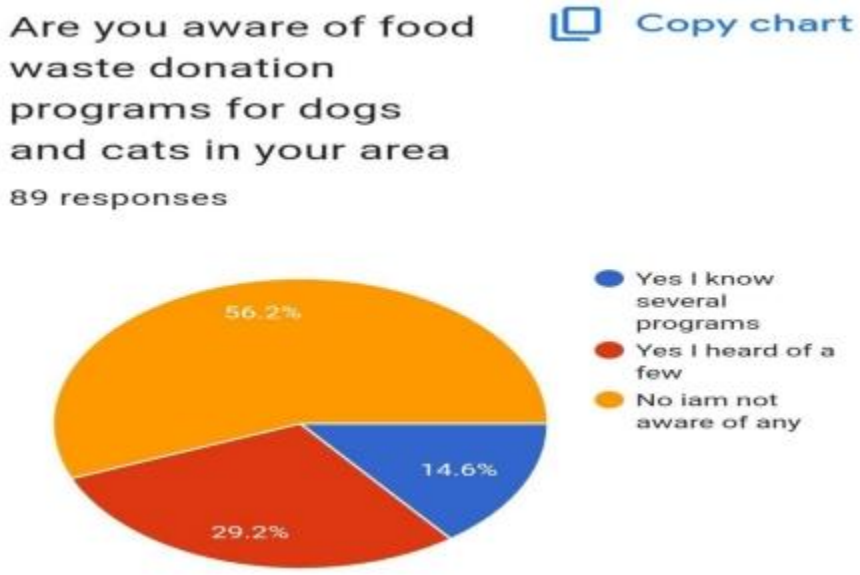
The food waste management system for animals aims to reduce food waste by collecting and reusing leftover food to feed animals in a safe and efficient manner. We includes the collection of food waste from households, restaurants, and other sources. The waste is then sorted to remove harmful items, processed to ensure it is safe for animal consumption, and distributed to farms, shelters, or animal feeding centers. Our website serves as a single platform that connects users (food waste donors) with us, and we further connect them to relevant organizations like animal shelters, farms, or feed processors. This system not only minimizes environmental pollution but also provides a sustainable and cost-effective food source for animals, promoting better waste management practices.

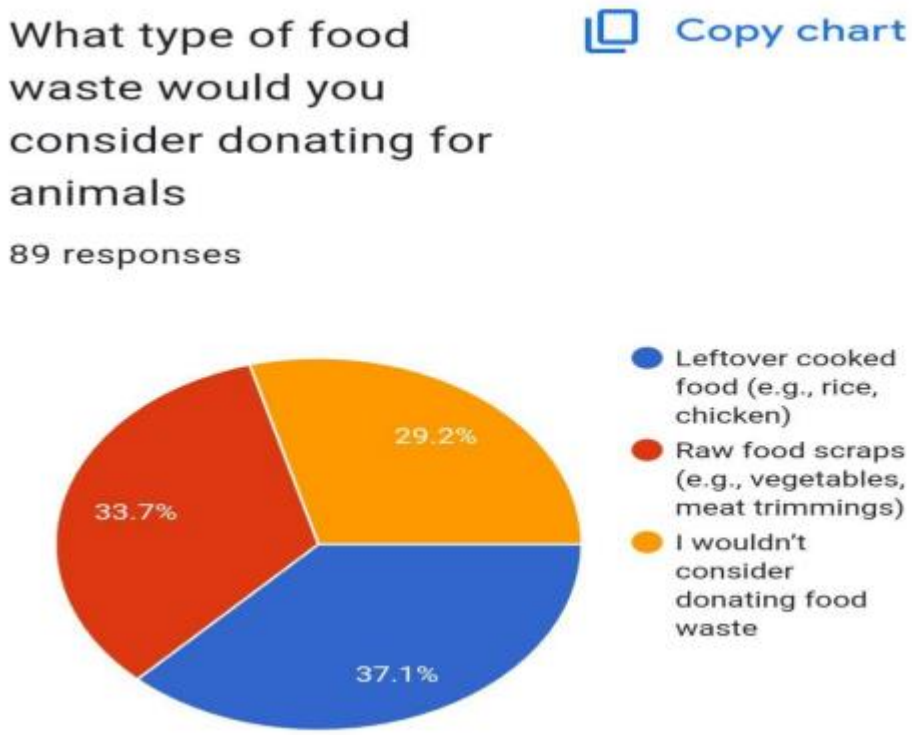
System Analysis

Study of Existing System

Existing System Currently, there are several organizations and platforms addressing food waste management, mostly focusing on human consumption but indirectly benefiting animals:
• Sodexo India & Feeding India Partnership: Redistributes surplus food to underserved communities but indirectly benefits animals by reducing overall waste.
• DONATEinKIND: Connects food donors to NGOs and food banks but lacks an animal-specific focus.
• Wastelink: Processes surplus food into animal feed but primarily operates on a business-model.
• Feeding America: Focuses on surplus food redistribution across the U.S., emphasizing human welfare, with little attention to animal feed management.
• Robin Hood Army :A volunteer-based organization redistributing surplus food to the needy, with no provisions for animal welfare.
• Poultry Feed Innovations: Limited initiatives exist to turn leftover grains and bread into poultry feed, which are underutilized.
• Agricultural Waste Projects: Some initiatives process agricultural byproducts into animal feed, but scalability remains a challenge.
• International Efforts (e.g., Too Good To Go): Primarily focus on reducing food waste but fail to incorporate animal welfare in their frameworks.
• Research Projects: Universities and research institutions propose models for food waste-to animal feed conversion but lack practical implementation. Despite these efforts, gaps persist in addressing animal-specific needs and optimizing waste management for animal welfare.

Literary Survey

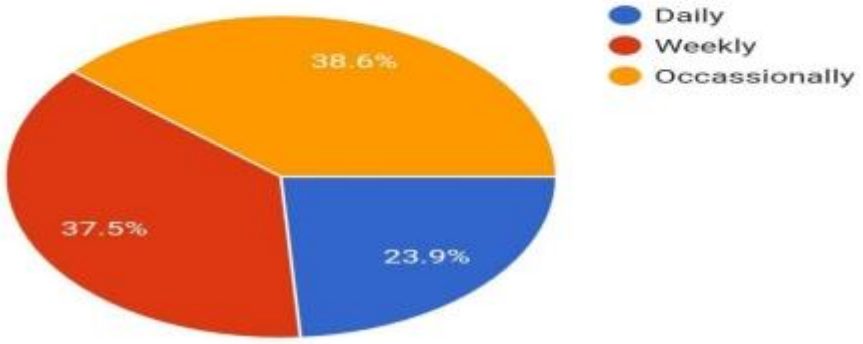




How often would you be willing to donate food waste for dogs and cats

88 responses

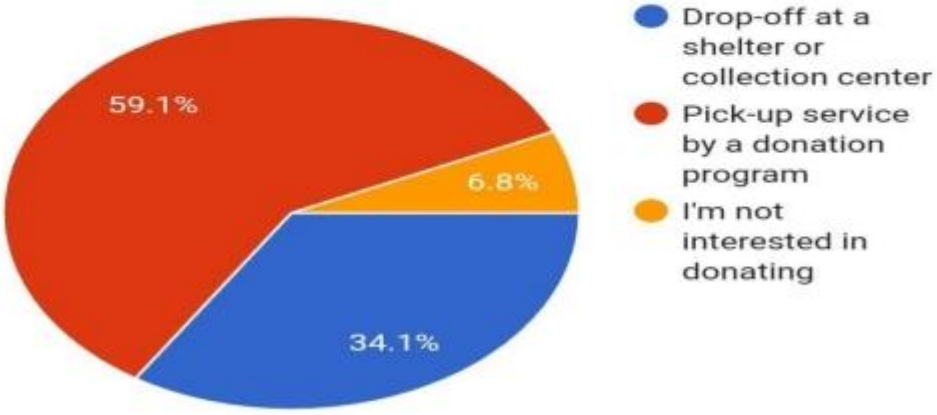
Copy chart



How would you prefer to donate food waste

88 responses

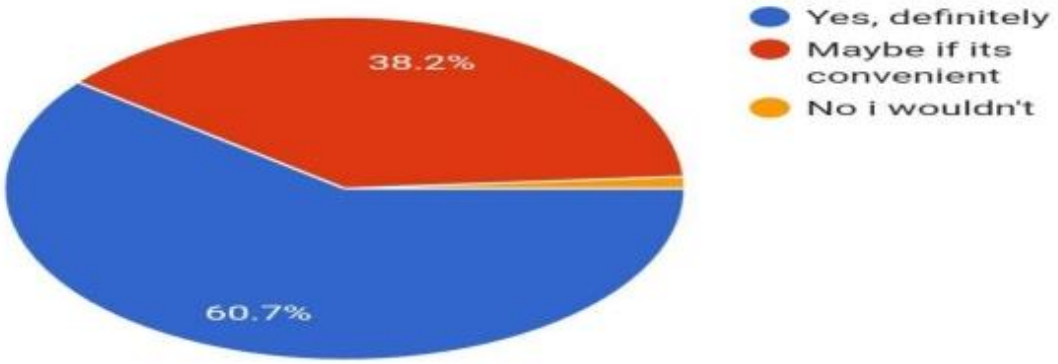
Copy chart



Would you participate in a food waste donation program if it was available nearby

Copy chart

89 responses



Interpretation

The survey highlights the need of a platform where people can donate the leftovers and make donations towards animals .By making it easy for the people to donate, the website appears to be a go to solution for animal lovers around the country .with the option of the donation being picked from home and also having the options to donate the kind of food people want to the website offers a wide range of options for people to work towards the cause of touching lives.

Problem Definition

Problem Definition Existing food waste management systems face several limitations that hinder their effectiveness, especially in addressing animal needs:

• Human-Centric Models: Most systems cater exclusively to human consumption, sidelining animal welfare.

• Logistical Challenges: Inefficient collection and distribution often lead to delays and spoilage of food waste.

• Lack of Awareness: Many individuals and businesses are unaware of how to repurpose food waste for animals.

• Inadequate Infrastructure: Insufficient facilities for processing food waste into animal feed.

• High Costs: Current systems have high operational costs, making them unsustainable for small-scale donors.

• Waste Segregation Issues: Difficulty in sorting food waste suitable for animal feed versus compost or disposal.

• Post-Consumer Waste Neglect: Minimal efforts are made to utilize food waste generated by households.

• Regulatory Challenges: Lack of clear guidelines for using processed food waste as animal feed.

• Environmental Impact: Food waste in landfills emits methane, contributing to climate change, which existing systems fail to address adequately.

• Limited Collaboration: Lack of partnerships between food businesses, charities, and animal welfare organizations, preventing more efficient redistribution of food waste.

• Health and Safety Concerns: Fear of food borne illnesses or contamination may deter the use of food waste for animal consumption, despite potential benefits.

• Public Perception: Negative perceptions of food waste reuse, whether for animals or composting, hinder participation from the public and businesses alike.

• Seasonal Variability: Fluctuating availability of certain types of food waste depending on the season, complicating the consistent feed supply for animals. Addressing these challenges is critical to creating a more effective and inclusive food waste management system.

Proposed System

Proposed System The proposed system is designed to bridge the gap between food waste and animal welfare by introducing advanced solutions:

• Centralized Platform for Users: Our website serves as a single platform that connects users (food waste donors) with us, and we further connect them to relevant organizations like animal shelters, farms, or feed processors.

• Data Collection and Management: The system collects data from users, including details of donors (e.g., restaurants, households) and organizations (e.g., shelters, farms). This data is stored securely in database.

• HTML and CSS Integration: The website is built using HTML and CSS to ensure a user-friendly, interactive, and visually appealing interface that simplifies the donor experience.

• Database Storage and Cleansing: All collected data is stored in a robust database (using SQL). The data is cleansed, verified, and organized before proceeding with further actions like allocation or distribution

• Efficient Data Processing: The backend system processes and analyzes the data to ensure that the donations are directed to organizations with the most immediate needs.

• User Accounts and History: Donors and organizations can create accounts to track their contributions or received donations, promoting transparency and engagement.

• Feedback Mechanism: Provides a feedback loop where users and organizations can review the process, ensuring continuous improvement.

• Automated Notifications: Sends reminders/Mails and alerts to users about donation schedules, updates on food collection, or status of their donations.

• Scalability: Designed to accommodate an increasing number of users and organizations as the platform grows, ensuring long-term sustainability

Objective of the System Proposed

The primary objectives of this project are to:

• Create a seamless digital platform connecting food waste generators with animal feed producers.

• Develop scalable solutions adaptable to diverse regions and community needs.

• Incorporate advanced technologies like geolocation and tracking for efficient logistics.

• Ensure proper classification and processing of food waste to meet the dietary needs of animals.

• Minimize food waste's environmental impact by diverting it from landfills to productive use.

• Engage with local communities and organizations for better outreach and collaboration.

• Establish long-term sustainability through circular economy practices

Requirement Specification

Requirement Specification
System Specification:

Hardware Specification

Processor: 1 GHz or faster

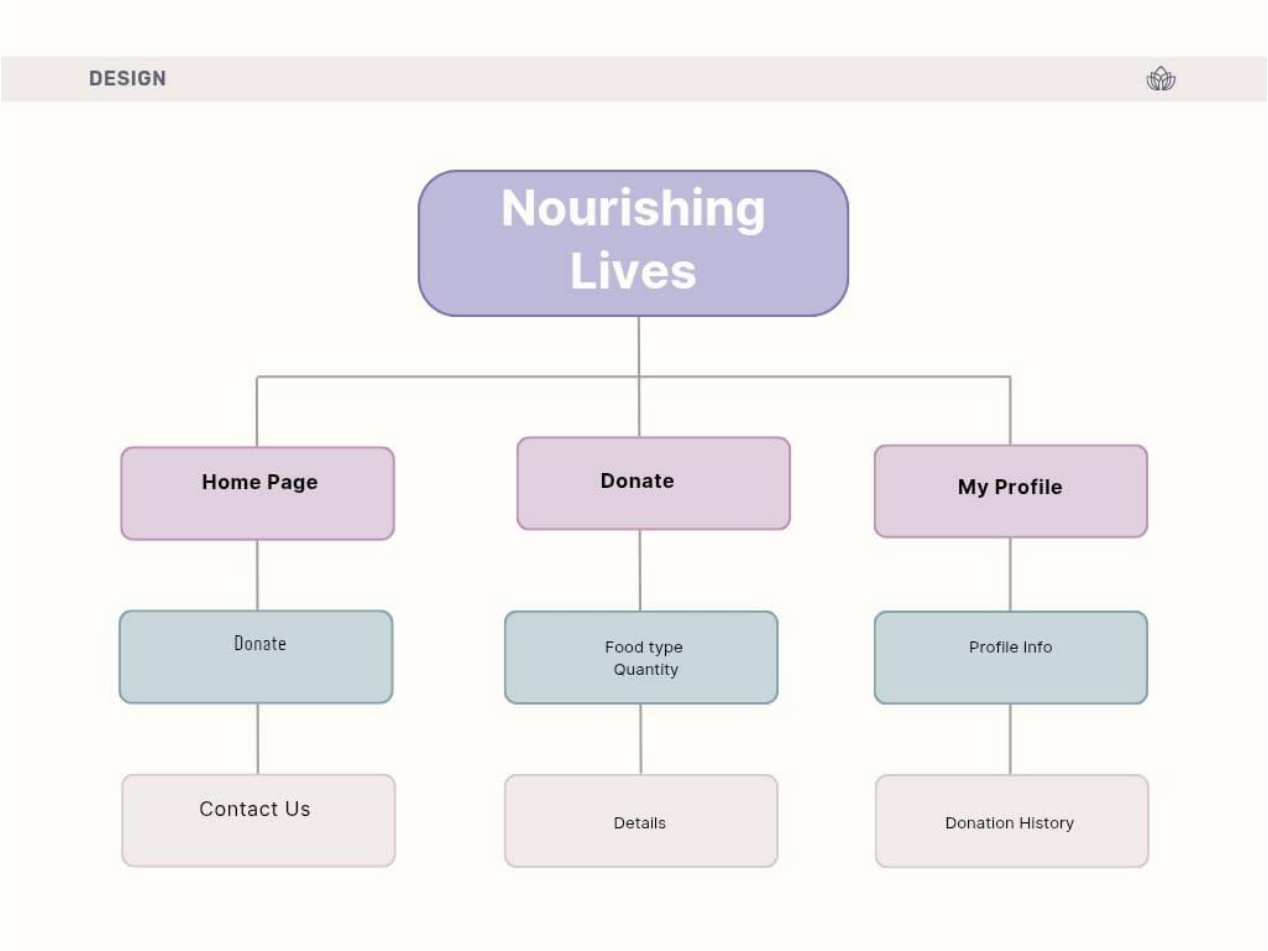
- RAM: 2 GB minimum, 4 GB recommended
- Disk Space: 100 MB for application, plus space for database growth
- Display: 1024x768 resolution or higher

Software Specification

- Operating System: Windows 7/8/10/11, macOS 10.13+, or Linux
- Python 3.6 or higher (3.8+ recommended)
- Modern web browser (Chrome, Firefox, Edge, Safari)
- Internet connection for initial setup only (to download dependencies)
- TECHNOLOGY STACK
- Backend Technologies
- 1. Python 3.8: Core programming language
- 2. Flask 2.3.3: Web framework for handling HTTP requests and responses
- 3. SQLite 3: Local database engine (built into Python)
- 4. Werkzeug 2.3.7: WSGI utility library for web applications
- Used for password hashing and secure file uploads
- 5. Gunicorn 23.0.0: WSGI HTTP Server for production deployment
- Frontend Technologies
- 1. HTML5: Markup language for website structure
- 2. CSS3: Styling language for appearance
- 3. Bootstrap 5: CSS framework for responsive design
- - Loaded from CDN: cdn.replit.com/agent/bootstrap-agent-dark-theme.min.css
- 4. JavaScript (ES6+): Programming language for client-side functionality
- 5. Font Awesome 6.4.0: Icon library for UI elements
- - Loaded from CDN: cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css

Design

Design



Explanation:

- The user has to register/login from the login page
- The donation page consists a list of items the user can choose from, the quantity ,the time and location
- Then the user will have to submit the donation form.
- The donation will be submitted and a volunteer will pick it up from the user as per the time and location specified.
- The user can also check his donation history and track his donation from the 'My Profile' page.

Coding

Coding
HTML File(index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Animal Food Waste Management</title>
   <link rel="stylesheet" href="https://cdn.replit.com/agent/bootstrap-agent-dark-theme.min.css">
   <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
   <link rel="stylesheet" href="{{ url_for('static', filename='css/styles.css') }}">
</head>
<body>
   <header>
      <div class="container">
         <div class="logo">
            <h1><i class="fas fa-paw"></i> Nourishing Lives</h1>
            <p>Food Waste Management for Animals</p>
         </div>
         <nav>
            <ul>
               <li><a href="#home" class="active">Home</a></li>
               <li><a href="#about">About</a></li>
               <li><a href="#donate">Donate</a></li>
               <li><a href="#profile" class="profile-link hidden">My Profile</a></li>
               <li><a href="#" class="login-btn">Login</a></li>
               <li><a href="#" class="logout-btn hidden">Logout</a></li>
            </ul>
         </nav>
      </div>
   </header>

   <main>
      <!-- Home Section -->
      <section id="home" class="hero">
         <div class="container">
            <div class="hero-content">
               <h2>Reducing Food Waste, Feeding Animals in Need</h2>
               <p>Connect with animal care facilities to donate your excess food and help feed
animals in need.</p>
               <div class="hero-buttons">
                  <button class="btn btn-primary" onclick="scrollToDonate()">Donate
Now</button>
                  <button class="btn btn-secondary login-btn">Join Us</button>
               </div>
            </div>
            <div class="hero-image">
               <img src="https://images.unsplash.com/photo-1535930891776-
0c2dfb7fda1a?q=80&w=500&auto=format&fit=crop" alt="Animals">
```

```
            </div>
        </div>
    </section>

    <!-- About Section -->
    <section id="about" class="about">
        <div class="container">
            <h2>About Our Mission</h2>
            <div class="about-content">
                <div class="about-text">
                    <p>At Nourishing Lives, we believe in creating a sustainable ecosystem where
food waste is minimized and repurposed to help animals in need. Our platform connects
restaurants, grocery stores, and individuals with animal shelters, zoos, farms, and wildlife
rehabilitation centers.</p>
                    <p>By donating your excess food, you're not only reducing waste but also
providing nutritious meals to animals and helping facilities reduce their food costs.</p>
                    <h3>How It Works</h3>
                    <ul>
                        <li><i class="fas fa-user-plus"></i> Create an account to get started</li>
                        <li><i class="fas fa-clipboard-list"></i> Submit details about your food
donation</li>
                        <li><i class="fas fa-truck"></i> Arrange pickup or delivery</li>
                        <li><i class="fas fa-check-circle"></i> Track your contribution impact</li>
                    </ul>
                </div>
                <div class="stats-container">
                    <div class="stat-box">
                        <i class="fas fa-donate"></i>
                        <h3>Food Donated</h3>
                        <p><span id="donations-count">0</span> donations</p>
                    </div>
                    <div class="stat-box">
                        <i class="fas fa-users"></i>
                        <h3>Active Users</h3>
                        <p>Join our growing community</p>
                    </div>
                    <div class="stat-box">
                        <i class="fas fa-globe-americas"></i>
                        <h3>Impact</h3>
                        <p>Reducing waste, feeding animals</p>
                    </div>
                </div>
            </div>
        </div>
    </section>

    <!-- Donation Section -->
    <section id="donate" class="donate">
        <div class="container">
            <h2>Donate Food</h2>
            <div class="donation-content">
                <div class="auth-required-message">
```

```
                <p>Please <a href="#" class="login-btn">login</a> or <a href="#" class="register-
btn">register</a> to make a donation.</p>
            </div>
            <form id="donation-form" class="hidden">
                <div class="form-group">
                    <label for="food-type">Food Type</label>
                    <select id="food-type" required>
                        <option value="">Select food type</option>
                        <option value="Fruits and Vegetables">Fruits and Vegetables</option>
                        <option value="Meat Scraps">Meat Scraps</option>
                        <option value="Bread and Grains">Bread and Grains</option>
                        <option value="Dairy Products">Dairy Products</option>
                        <option value="Pet Food">Pet Food</option>
                        <option value="Mixed Food Waste">Mixed Food Waste</option>
                        <option value="Other">Other (specify in notes)</option>
                    </select>
                </div>
                <div class="form-group">
                    <label for="quantity">Quantity</label>
                    <input type="text" id="quantity" placeholder="e.g., 5 kg, 3 bags" required>
                </div>
                <div class="form-group">
                    <label for="location">Location</label>
                    <select id="location" required>
                        <option value="">Select your city</option>
                        <!-- Cities will be populated by JavaScript -->
                    </select>
                </div>
                <div class="form-group">
                    <label for="pickup-date">Preferred Pickup Date/Time (Optional)</label>
                    <input type="datetime-local" id="pickup-date">
                </div>
                <div class="form-group">
                    <label for="notes">Additional Notes (Optional)</label>
                    <textarea id="notes" placeholder="Any special instructions or details about the
food"></textarea>
                </div>
                <button type="submit" class="btn btn-primary">Submit Donation</button>
            </form>
        </div>
    </div>
</section>

<!-- Profile Section -->
<section id="profile" class="profile hidden">
    <div class="container">
        <h2>My Profile</h2>
        <div class="profile-tabs">
            <button class="tab-btn active" data-tab="profile-info">Profile Info</button>
            <button class="tab-btn" data-tab="donation-history">Donation History</button>
            <button class="tab-btn" data-tab="account-settings">Account Settings</button>
        </div>
```

```
<!-- Profile Info Tab -->
<div class="tab-content profile-info active">
  <div class="profile-card">
    <div class="profile-image">
      <img id="profile-image" src="{{ url_for('static', filename='uploads/default-
profile.png') }}" alt="Profile Image">
      <label for="profile-image-upload" class="image-upload-btn">
        <i class="fas fa-camera"></i>
      </label>
      <input type="file" id="profile-image-upload" accept="image/*" style="display:
none;">
    </div>
    <div class="profile-details">
      <h3 id="profile-name">Loading...</h3>
      <p><i class="fas fa-user"></i> <span id="profile-
username">Loading...</span></p>
      <p><i class="fas fa-envelope"></i> <span id="profile-
email">Loading...</span></p>
      <p><i class="fas fa-phone"></i> <span id="profile-
contact">Loading...</span></p>
      <p><i class="fas fa-map-marker-alt"></i> <span id="profile-
city">Loading...</span></p>
    </div>
  </div>
  <div class="profile-update">
    <h3>Update Profile Information</h3>
    <form id="profile-update-form">
      <div class="form-group">
        <label for="update-name">Name</label>
        <input type="text" id="update-name" placeholder="Your full name">
      </div>
      <div class="form-group">
        <label for="update-email">Email</label>
        <input type="email" id="update-email" placeholder="Your email address">
      </div>
      <div class="form-group">
        <label for="update-contact">Phone/Alternative Contact</label>
        <input type="text" id="update-contact" placeholder="Phone number or
alternative contact">
      </div>
      <div class="form-group">
        <label for="update-city">Preferred City</label>
        <select id="update-city">
          <option value="">Select your city</option>
          <!-- Cities will be populated by JavaScript -->
        </select>
      </div>
      <button type="submit" class="btn btn-primary">Update Profile</button>
    </form>
  </div>
</div>
```

```
            <!-- Donation History Tab -->
            <div class="tab-content donation-history">
                <div class="donation-filters">
                    <button class="filter-btn active" data-filter="all">All Donations</button>
                    <button class="filter-btn" data-filter="pending">Pending</button>
                    <button class="filter-btn" data-filter="completed">Completed</button>
                    <button class="filter-btn" data-filter="cancelled">Cancelled</button>
                </div>
                <div class="donations-container" id="donations-container">
                    <!-- Donations will be populated by JavaScript -->
                    <p class="loading-message">Loading your donation history...</p>
                </div>
            </div>

            <!-- Account Settings Tab -->
            <div class="tab-content account-settings">
                <div class="settings-card">
                    <h3>Change Password</h3>
                    <form id="password-update-form">
                        <div class="form-group">
                            <label for="current-password">Current Password</label>
                            <input type="password" id="current-password" required>
                        </div>
                        <div class="form-group">
                            <label for="new-password">New Password</label>
                            <input type="password" id="new-password" required>
                            <div class="password-strength">
                                <div class="strength-meter"></div>
                                <p class="strength-text">Password strength</p>
                            </div>
                        </div>
                        <div class="form-group">
                            <label for="confirm-password">Confirm New Password</label>
                            <input type="password" id="confirm-password" required>
                        </div>
                        <button type="submit" class="btn btn-primary">Update Password</button>
                    </form>
                </div>
            </div>
        </div>
    </section>

    <!-- Admin Section (hidden by default) -->
    <section id="admin" class="admin hidden">
        <div class="container">
            <h2>Admin Dashboard</h2>
            <div class="admin-tabs">
                <button class="tab-btn active" data-tab="admin-donations">Manage
Donations</button>
                <button class="tab-btn" data-tab="admin-cities">Manage Cities</button>
                <button class="tab-btn" data-tab="admin-users">Manage Users</button>
```

```
        </div>

        <!-- Admin Donations Tab -->
        <div class="tab-content admin-donations active">
            <h3>All Donations</h3>
            <div class="admin-filters">
                <button class="filter-btn active" data-filter="all">All</button>
                <button class="filter-btn" data-filter="pending">Pending</button>
                <button class="filter-btn" data-filter="completed">Completed</button>
                <button class="filter-btn" data-filter="cancelled">Cancelled</button>
            </div>
            <div class="admin-donations-list" id="admin-donations-list">
                <!-- Donations will be populated by JavaScript -->
                <p class="loading-message">Loading donations...</p>
            </div>
        </div>

        <!-- Admin Cities Tab -->
        <div class="tab-content admin-cities">
            <h3>Manage Cities</h3>
            <div class="admin-add-form">
                <form id="city-add-form">
                    <div class="form-group">
                        <label for="city-name">City Name</label>
                        <input type="text" id="city-name" placeholder="Enter city name" required>
                    </div>
                    <button type="submit" class="btn btn-primary">Add City</button>
                </form>
            </div>
            <div class="admin-city-list" id="admin-city-list">
                <!-- Cities will be populated by JavaScript -->
                <p class="loading-message">Loading cities...</p>
            </div>
        </div>

        <!-- Admin Users Tab -->
        <div class="tab-content admin-users">
            <h3>Manage Users</h3>
            <div class="admin-user-list" id="admin-user-list">
                <!-- Users will be populated by JavaScript -->
                <p class="loading-message">Loading users...</p>
            </div>
        </div>
    </div>
</section>

<!-- Owner Section (hidden by default) -->
<section id="owner" class="admin hidden">
    <div class="container">
        <h2>Owner Dashboard</h2>
        <div class="owner-message">
            <div class="alert alert-info">
```

```
                <p><strong>Welcome Owner!</strong> You have full control over the
system.</p>
            </div>
        </div>
        <div class="admin-tabs">
            <button class="tab-btn active" data-tab="owner-donations">All Donations</button>
            <button class="tab-btn" data-tab="owner-users">Manage Users</button>
            <button class="tab-btn" data-tab="owner-stats">Statistics</button>
        </div>

        <!-- Owner Donations Tab -->
        <div class="tab-content owner-donations active">
            <h3>Donation Management</h3>
            <div class="admin-filters">
                <button class="filter-btn active" data-filter="all">All</button>
                <button class="filter-btn" data-filter="pending">Pending</button>
                <button class="filter-btn" data-filter="completed">Completed</button>
                <button class="filter-btn" data-filter="cancelled">Cancelled</button>
            </div>
            <div class="admin-donations-list" id="owner-donations-list">
                <!-- Donations will be populated by JavaScript -->
                <p class="loading-message">Loading all donations...</p>
            </div>
        </div>

        <!-- Owner Users Tab -->
        <div class="tab-content owner-users">
            <h3>User Management</h3>
            <div class="owner-user-controls">
                <div class="form-group">
                    <input type="text" id="user-search" placeholder="Search users...">
                </div>
            </div>
            <div class="admin-user-list" id="owner-user-list">
                <!-- Users will be populated by JavaScript -->
                <p class="loading-message">Loading all users...</p>
            </div>
        </div>

        <!-- Owner Stats Tab -->
        <div class="tab-content owner-stats">
            <h3>System Statistics</h3>
            <div class="stats-grid">
                <div class="stat-card">
                    <h4>User Statistics</h4>
                    <p>Total Users: <span id="stat-total-users">0</span></p>
                    <p>Admins: <span id="stat-admins">0</span></p>
                    <p>Regular Users: <span id="stat-regular-users">0</span></p>
                </div>
                <div class="stat-card">
                    <h4>Donation Statistics</h4>
                    <p>Total Donations: <span id="stat-total-donations">0</span></p>
```

```
                <p>Pending: <span id="stat-pending-donations">0</span></p>
                <p>Completed: <span id="stat-completed-donations">0</span></p>
                <p>Cancelled: <span id="stat-cancelled-donations">0</span></p>
            </div>
            <div class="stat-card">
                <h4>Location Statistics</h4>
                <p>Most Active City: <span id="stat-most-active-city">-</span></p>
                <p>Total Cities: <span id="stat-total-cities">0</span></p>
            </div>
        </div>
        <div class="recent-activity">
            <h4>Recent Activity</h4>
            <div id="activity-log">
                <p class="loading-message">Loading activity...</p>
            </div>
        </div>
    </div>
  </div>
 </section>
</main>

<footer>
 <div class="container">
  <div class="footer-content">
   <div class="footer-logo">
    <h2><i class="fas fa-paw"></i> Nourishing Lives</h2>
    <p>Food Waste Management for Animals</p>
   </div>
   <div class="footer-links">
    <h3>Quick Links</h3>
    <ul>
     <li><a href="#home">Home</a></li>
     <li><a href="#about">About</a></li>
     <li><a href="#donate">Donate</a></li>
     <li><a href="#" class="privacy-link">Privacy Policy</a></li>
     <li><a href="#" class="terms-link">Terms of Service</a></li>
    </ul>
   </div>
   <div class="footer-contact">
    <h3>Contact Us</h3>
    <p><i class="fas fa-envelope"></i> info@nourishinglives.org</p>
    <p><i class="fas fa-phone"></i> +91 88869-84800</p>
    <p><i class="fas fa-map-marker-alt"></i>Hyderabad, Telangana, India</p>
    <div class="social-links">
     <a href="#"><i class="fab fa-facebook-f"></i></a>
     <a href="#"><i class="fab fa-twitter"></i></a>
     <a href="#"><i class="fab fa-instagram"></i></a>
     <a href="#"><i class="fab fa-linkedin-in"></i></a>
    </div>
   </div>
  </div>
  <div class="footer-bottom">
```

```
            <p>&copy; 2025 Nourishing Lives. All rights reserved.</p>
        </div>
    </div>
</footer>

<!-- Modals -->
<!-- Login Modal -->
<div id="login-modal" class="modal">
    <div class="modal-content">
        <span class="close-btn">&times;</span>
        <h2>Login</h2>
        <form id="login-form">
            <div class="form-group">
                <label for="login-username">Username</label>
                <input type="text" id="login-username" required>
            </div>
            <div class="form-group">
                <label for="login-password">Password</label>
                <input type="password" id="login-password" required>
            </div>
            <div class="form-message"></div>
            <button type="submit" class="btn btn-primary">Login</button>
        </form>
        <div class="modal-footer-text">
            <p>Don't have an account? <a href="#" id="register-link">Register</a></p>
            <p><a href="#" id="forgot-password-link">Forgot Password?</a></p>
        </div>
    </div>
</div>

<!-- Register Modal -->
<div id="register-modal" class="modal">
    <div class="modal-content">
        <span class="close-btn">&times;</span>
        <h2>Create an Account</h2>
        <form id="register-form">
            <div class="form-group">
                <label for="register-username">Username</label>
                <input type="text" id="register-username" required>
                <p class="help-text">At least 3 characters, letters, numbers, and underscores
only.</p>
            </div>
            <div class="form-group">
                <label for="register-password">Password</label>
                <input type="password" id="register-password" required>
                <div class="password-strength">
                    <div class="strength-meter"></div>
                    <p class="strength-text">Password strength</p>
                </div>
                <p class="help-text">At least 8 characters.</p>
            </div>
            <div class="form-group">
```

```
                <label for="register-confirm-password">Confirm Password</label>
                <input type="password" id="register-confirm-password" required>
            </div>
            <div class="form-group">
                <label for="register-email">Email (Optional)</label>
                <input type="email" id="register-email">
            </div>
            <div class="form-group">
                <label for="register-name">Name (Optional)</label>
                <input type="text" id="register-name">
            </div>
            <div class="form-group">
                <label for="register-contact">Phone/Email (Optional)</label>
                <input type="text" id="register-contact">
            </div>
            <div class="form-group">
                <label for="register-city">Preferred City (Optional)</label>
                <select id="register-city">
                    <option value="">Select your city</option>
                    <!-- Cities will be populated by JavaScript -->
                </select>
            </div>
            <div class="form-message"></div>
            <button type="submit" class="btn btn-primary">Register</button>
        </form>
        <div class="modal-footer-text">
            <p>Already have an account? <a href="#" id="login-link">Login</a></p>
        </div>
    </div>
</div>

<!-- Forgot Password Modal -->
<div id="forgot-password-modal" class="modal">
    <div class="modal-content">
        <span class="close-btn">&times;</span>
        <h2>Forgot Password</h2>
        <form id="forgot-password-form">
            <div class="form-group">
                <label for="forgot-username">Username or Email</label>
                <input type="text" id="forgot-username" required>
            </div>
            <div class="form-message"></div>
            <button type="submit" class="btn btn-primary">Reset Password</button>
        </form>
        <div class="modal-footer-text">
            <p><a href="#" id="back-to-login">Back to Login</a></p>
        </div>
    </div>
</div>

<script src="{{ url_for('static', filename='js/script.js') }}"></script>
</body>
```

</html>

CSS File(styles1.css)

```
/* Base Styles */
:root {
    --primary-color: #3498db;
    --secondary-color: #2ecc71;
    --background-color: #f5f5f5;
    --text-color: #f5f5f5;
    --light-gray: #333;
    --medium-gray: #333;
    --dark-gray: #333;
    --box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    --border-radius: 5px;
    --transition: all 0.3s ease;
}

/* General Styles */
body {
    font-family: 'Inter', sans-serif;
    line-height: 1.6;
    color: var(--text-color);
    margin: 0;
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}

.container {
    width: 90%;
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 15px;
}

/* Header */
header {
    background-color: var(--bs-dark);
    color: white;
    padding: 1rem 0;
    box-shadow: var(--box-shadow);
}

.logo h1 {
    font-size: 1.8rem;
    margin: 0;
    color: var(--bs-info);
}

.logo p {
    margin: 0;
    font-size: 0.9rem;
```

```
    opacity: 0.8;
}

header .container {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

nav ul {
    display: flex;
    list-style: none;
    margin: 0;
    padding: 0;
}

nav ul li {
    margin-left: 1.5rem;
}

nav ul li a {
    color: white;
    text-decoration: none;
    font-weight: 500;
    transition: var(--transition);
}

nav ul li a:hover,
nav ul li a.active {
    color: var(--bs-info);
}

/* Hero Section */
.hero {
    padding: 4rem 0;
    background-color: var(--bs-dark);
    color: white;
}

.hero .container {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 2rem;
}

.hero-content {
    flex: 1;
}

.hero-image {
    flex: 1;
```

```
    text-align: center;
}

.hero-image img {
    max-width: 100%;
    height: auto;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.3);
}

.hero h2 {
    font-size: 2.5rem;
    margin-bottom: 1rem;
    color: var(--bs-info);
}

.hero p {
    font-size: 1.1rem;
    margin-bottom: 2rem;
}

.hero-buttons {
    display: flex;
    gap: 1rem;
}

/* About Section */
.about {
    padding: 4rem 0;
    background-color: var(--bs-body-bg);
}

.about h2 {
    text-align: center;
    margin-bottom: 2rem;
    color: var(--bs-info);
}

.about-content {
    display: flex;
    gap: 2rem;
}

.about-text {
    flex: 2;
}

.about-text ul {
    list-style: none;
    padding: 0;
}
```

```
.about-text ul li {
    margin-bottom: 0.8rem;
    display: flex;
    align-items: center;
}

.about-text ul li i {
    color: var(--bs-success);
    margin-right: 0.5rem;
}

.stats-container {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 1rem;
}

.stat-box {
    background-color: var(--bs-dark);
    border-radius: var(--border-radius);
    padding: 1.5rem;
    text-align: center;
    box-shadow: var(--box-shadow);
    color: white;
}

.stat-box i {
    font-size: 2rem;
    color: var(--bs-success);
    margin-bottom: 0.5rem;
}

.stat-box h3 {
    margin: 0.5rem 0;
    font-size: 1.3rem;
    color: var(--bs-info);
}

/* Donation Section */
.donate {
    padding: 4rem 0;
    background-color: var(--bs-tertiary-bg);
}

.donate h2 {
    text-align: center;
    margin-bottom: 2rem;
    color: var(--bs-info);
}

.donation-content {
```

```
    max-width: 600px;
    margin: 0 auto;
    background-color: var(--bs-dark);
    padding: 2rem;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
}

.auth-required-message {
    text-align: center;
    padding: 1rem;
}

.auth-required-message a {
    color: var(--bs-success);
    text-decoration: none;
    font-weight: bold;
}

.auth-required-message a:hover {
    text-decoration: underline;
}

/* Forms */
.form-group {
    margin-bottom: 1.5rem;
}

.form-group label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: 500;
}

.form-group input,
.form-group select,
.form-group textarea {
    width: 100%;
    padding: 0.8rem;
    border: 1px solid var(--bs-gray-700);
    border-radius: var(--border-radius);
    background-color: var(--bs-dark);
    color: white;
}

.form-group input:focus,
.form-group select:focus,
.form-group textarea:focus {
    outline: none;
    border-color: var(--bs-info);
}
```

```
.form-group textarea {
    min-height: 100px;
    resize: vertical;
}

.help-text {
    font-size: 0.8rem;
    color: var(--bs-info);
    margin-top: 0.3rem;
}

/* Profile Section */
.profile {
    padding: 4rem 0;
    background-color: var(--bs-body-bg);
}

.profile h2 {
    text-align: center;
    margin-bottom: 2rem;
    color: var(--bs-info);
}

.profile-tabs {
    display: flex;
    justify-content: center;
    margin-bottom: 2rem;
    border-bottom: 1px solid var(--bs-gray-700);
}

.tab-btn {
    background: none;
    border: none;
    padding: 0.8rem 1.5rem;
    cursor: pointer;
    font-weight: 500;
    color: var(--bs-light);
    transition: var(--transition);
    border-bottom: 3px solid transparent;
}

.tab-btn:hover {
    color: var(--bs-info);
}

.tab-btn.active {
    color: var(--bs-info);
    border-bottom-color: var(--bs-info);
}

.tab-content {
```

```
    display: none;
}

.tab-content.active {
    display: block;
}

.profile-card {
    display: flex;
    gap: 2rem;
    margin-bottom: 2rem;
    padding: 1.5rem;
    background-color: var(--bs-dark);
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
}

.profile-image {
    position: relative;
    width: 150px;
    height: 150px;
}

.profile-image img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    border-radius: 50%;
    border: 3px solid var(--bs-info);
}

.image-upload-btn {
    position: absolute;
    bottom: 0;
    right: 0;
    width: 40px;
    height: 40px;
    background-color: var(--bs-info);
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    transition: var(--transition);
}

.image-upload-btn:hover {
    background-color: var(--bs-primary);
}

.profile-details {
```

```
    flex: 1;
}

.profile-details h3 {
    margin-top: 0;
    margin-bottom: 1rem;
    color: var(--bs-info);
}

.profile-details p {
    margin: 0.5rem 0;
}

.profile-details p i {
    color: var(--bs-success);
    margin-right: 0.5rem;
    width: 20px;
    text-align: center;
}

.profile-update {
    background-color: var(--bs-dark);
    padding: 1.5rem;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
}

.profile-update h3 {
    margin-top: 0;
    margin-bottom: 1.5rem;
    color: var(--bs-info);
}

/* Password Strength Meter */
.password-strength {
    margin-top: 0.5rem;
}

.strength-meter {
    height: 5px;
    background-color: var(--bs-gray-700);
    border-radius: 3px;
    overflow: hidden;
    position: relative;
}

.strength-meter::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
```

```
    height: 100%;
    width: 0;
    transition: var(--transition);
}

.strength-meter.weak::before {
    width: 33.3%;
    background-color: var(--bs-danger);
}

.strength-meter.medium::before {
    width: 66.6%;
    background-color: var(--bs-warning);
}

.strength-meter.strong::before {
    width: 100%;
    background-color: var(--bs-success);
}

.strength-text {
    font-size: 0.8rem;
    margin-top: 0.3rem;
    color: var(--bs-gray-400);
}

/* Donation History */
.donation-filters {
    display: flex;
    gap: 0.5rem;
    margin-bottom: 1.5rem;
}

.filter-btn {
    padding: 0.5rem 1rem;
    border: none;
    background-color: var(--bs-dark);
    color: white;
    border-radius: var(--border-radius);
    cursor: pointer;
    transition: var(--transition);
}

.filter-btn:hover {
    background-color: var(--bs-gray-700);
}

.filter-btn.active {
    background-color: var(--bs-info);
}

.donations-container {
```

```
    background-color: var(--bs-dark);
    padding: 1.5rem;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
}

.donation-card {
    background-color: var(--bs-gray-800);
    padding: 1rem;
    border-radius: var(--border-radius);
    margin-bottom: 1rem;
    border-left: 4px solid transparent;
}

.donation-card.pending {
    border-left-color: var(--bs-warning);
}

.donation-card.completed {
    border-left-color: var(--bs-success);
}

.donation-card.cancelled {
    border-left-color: var(--bs-danger);
}

.donation-header {
    display: flex;
    justify-content: space-between;
    margin-bottom: 0.5rem;
}

.donation-title {
    font-weight: bold;
    color: var(--bs-white);
}

.donation-status {
    padding: 0.2rem 0.5rem;
    border-radius: 3px;
    font-size: 0.8rem;
    font-weight: bold;
}

.donation-status.pending {
    background-color: var(--bs-warning);
    color: black;
}

.donation-status.completed {
    background-color: var(--bs-success);
```

```
    color: white;
}

.donation-status.cancelled {
    background-color: var(--bs-danger);
    color: white;
}

.donation-details {
    font-size: 0.9rem;
}

.donation-details p {
    margin: 0.3rem 0;
}

.donation-actions {
    margin-top: 1rem;
}

.donation-actions button {
    margin-right: 0.5rem;
}

/* Admin/Owner Section */
.admin {
    padding: 4rem 0;
    background-color: var(--bs-body-bg);
}

.admin h2 {
    text-align: center;
    margin-bottom: 0.5rem;
    color: var(--bs-info);
}

.owner-message {
    margin-bottom: 2rem;
}

.admin-filters {
    margin-bottom: 1.5rem;
}

.admin-donations-list,
.admin-user-list,
.admin-city-list {
    background-color: var(--bs-dark);
    padding: 1.5rem;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
```

```
}

.user-card,
.city-card {
    background-color: var(--bs-gray-800);
    padding: 1rem;
    border-radius: var(--border-radius);
    margin-bottom: 1rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.user-card.admin {
    border-left: 4px solid var(--bs-success);
}

.user-card.owner {
    border-left: 4px solid var(--bs-danger);
}

.user-info,
.city-info {
    flex: 1;
}

.user-actions,
.city-actions {
    display: flex;
    gap: 0.5rem;
}

.admin-add-form {
    margin-bottom: 2rem;
}

/* Stats Section */
.stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 1.5rem;
    margin-bottom: 2rem;
}

.stat-card {
    background-color: var(--bs-dark);
    padding: 1.5rem;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
}
```

```
.stat-card h4 {
    color: var(--bs-info);
    margin-top: 0;
    margin-bottom: 1rem;
    font-size: 1.2rem;
}

.stat-card p {
    margin: 0.5rem 0;
}

.recent-activity {
    background-color: var(--bs-dark);
    padding: 1.5rem;
    border-radius: var(--border-radius);
    box-shadow: var(--box-shadow);
    color: white;
}

.recent-activity h4 {
    color: var(--bs-info);
    margin-top: 0;
    margin-bottom: 1rem;
    font-size: 1.2rem;
}

.activity-item {
    padding: 0.8rem;
    border-bottom: 1px solid var(--bs-gray-700);
}

.activity-item:last-child {
    border-bottom: none;
}

.activity-header {
    display: flex;
    justify-content: space-between;
    margin-bottom: 0.3rem;
}

.activity-user {
    font-weight: bold;
    color: var(--bs-info);
}

.activity-time {
    font-size: 0.8rem;
    color: var(--bs-gray-400);
}

.activity-action {
```

```
    margin-bottom: 0.3rem;
}

.activity-details {
    font-size: 0.9rem;
    color: var(--bs-gray-300);
}

/* Footer */
footer {
    background-color: var(--bs-dark);
    color: white;
    padding: 3rem 0 1rem;
    margin-top: auto;
}

.footer-content {
    display: flex;
    flex-wrap: wrap;
    justify-content: space-between;
    gap: 2rem;
    margin-bottom: 2rem;
}

.footer-logo {
    flex: 1;
    min-width: 250px;
}

.footer-logo h2 {
    margin-top: 0;
    margin-bottom: 0.5rem;
    color: var(--bs-info);
    font-size: 1.5rem;
}

.footer-links {
    flex: 1;
    min-width: 200px;
}

.footer-links h3,
.footer-contact h3 {
    color: var(--bs-info);
    margin-bottom: 1rem;
    font-size: 1.2rem;
}

.footer-links ul {
    list-style: none;
    padding: 0;
    margin: 0;
```

```
}

.footer-links ul li {
    margin-bottom: 0.5rem;
}

.footer-links ul li a {
    color: var(--bs-light);
    text-decoration: none;
    transition: var(--transition);
}

.footer-links ul li a:hover {
    color: var(--bs-info);
}

.footer-contact {
    flex: 1;
    min-width: 250px;
}

.footer-contact p {
    margin: 0.5rem 0;
}

.footer-contact p i {
    color: var(--bs-success);
    margin-right: 0.5rem;
    width: 20px;
    text-align: center;
}

.social-links {
    display: flex;
    gap: 1rem;
    margin-top: 1rem;
}

.social-links a {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    width: 35px;
    height: 35px;
    background-color: var(--bs-gray-800);
    color: white;
    border-radius: 50%;
    transition: var(--transition);
}

.social-links a:hover {
    background-color: var(--bs-info);
```

```
    transform: translateY(-3px);
}

.footer-bottom {
    text-align: center;
    padding-top: 1.5rem;
    border-top: 1px solid var(--bs-gray-700);
    font-size: 0.9rem;
    color: var(--bs-gray-400);
}

/* Modals */
.modal {
    display: none;
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background-color: rgba(0, 0, 0, 0.7);
    z-index: 1000;
    overflow: auto;
}

.modal-content {
    background-color: var(--bs-dark);
    margin: 5% auto;
    padding: 2rem;
    border-radius: var(--border-radius);
    max-width: 500px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.3);
    position: relative;
    color: white;
}

.close-btn {
    position: absolute;
    top: 10px;
    right: 15px;
    color: var(--bs-gray-400);
    font-size: 1.5rem;
    cursor: pointer;
    transition: var(--transition);
}

.close-btn:hover {
    color: var(--bs-danger);
}

.modal h2 {
    text-align: center;
    margin-bottom: 1.5rem;
```

```
    color: var(--bs-info);
}

.modal-footer-text {
    text-align: center;
    margin-top: 1.5rem;
    font-size: 0.9rem;
}

.modal-footer-text a {
    color: var(--bs-info);
    text-decoration: none;
}

.modal-footer-text a:hover {
    text-decoration: underline;
}

.form-message {
    margin: 1rem 0;
    padding: 0.8rem;
    border-radius: var(--border-radius);
    display: none;
}

.form-message.error {
    display: block;
    background-color: var(--bs-danger);
    color: white;
}

.form-message.success {
    display: block;
    background-color: var(--bs-success);
    color: white;
}

.form-message.info {
    display: block;
    background-color: var(--bs-info);
    color: white;
}

/* Utility Classes */
.btn {
    display: inline-block;
    padding: 0.8rem 1.5rem;
    border: none;
    border-radius: var(--border-radius);
    cursor: pointer;
    font-weight: 500;
    text-decoration: none;
```

```
    transition: var(--transition);
}

.btn-primary {
    background-color: var(--bs-info);
    color: white;
}

.btn-primary:hover {
    background-color: var(--bs-primary);
}

.btn-secondary {
    background-color: var(--bs-secondary);
    color: white;
}

.btn-secondary:hover {
    background-color: var(--bs-gray-700);
}

.btn-danger {
    background-color: var(--bs-danger);
    color: white;
}

.btn-danger:hover {
    background-color: #c0392b;
}

.btn-warning {
    background-color: var(--bs-warning);
    color: black;
}

.btn-warning:hover {
    background-color: #f39c12;
}

.btn-success {
    background-color: var(--bs-success);
    color: white;
}

.btn-success:hover {
    background-color: #27ae60;
}

.btn-sm {
    padding: 0.4rem 0.8rem;
    font-size: 0.9rem;
}
```

```
.hidden {
    display: none;
}

.alert {
    padding: 1rem;
    border-radius: var(--border-radius);
    margin-bottom: 1rem;
}

.alert-info {
    background-color: var(--bs-info);
    color: white;
}

.alert-success {
    background-color: var(--bs-success);
    color: white;
}

.alert-warning {
    background-color: var(--bs-warning);
    color: black;
}

.alert-danger {
    background-color: var(--bs-danger);
    color: white;
}

.loading-message {
    text-align: center;
    padding: 1rem;
    color: var(--bs-gray-400);
}

/* Responsive Styles */
@media (max-width: 992px) {
    .hero .container {
        flex-direction: column;
    }

    .about-content {
        flex-direction: column;
    }

    .stats-container {
        flex-direction: row;
    }
}
```

```
@media (max-width: 768px) {
    header .container {
        flex-direction: column;
        gap: 1rem;
    }

    .logo {
        text-align: center;
    }

    nav ul {
        justify-content: center;
        flex-wrap: wrap;
    }

    nav ul li {
        margin: 0 0.5rem;
    }

    .hero h2 {
        font-size: 2rem;
    }

    .profile-card {
        flex-direction: column;
        align-items: center;
    }

    .profile-image {
        margin-bottom: 1rem;
    }

    .profile-details {
        text-align: center;
    }

    .stats-container {
        flex-direction: column;
    }

    .donation-header {
        flex-direction: column;
        gap: 0.5rem;
    }
}

@media (max-width: 576px) {
    .filter-btn, .tab-btn {
        font-size: 0.9rem;
        padding: 0.5rem 0.8rem;
    }
```

```
    .hero-buttons {
        flex-direction: column;
    }

    .footer-content {
        flex-direction: column;
        gap: 2rem;
    }

    .footer-logo, .footer-links, .footer-contact {
        text-align: center;
    }

    .social-links {
        justify-content: center;
    }
}
```

JavaScript File (script1.js)

```
/**
 * Animal Food Waste Management
 * Main JavaScript file
 */

document.addEventListener('DOMContentLoaded', function() {
    initApp();
});

/**
 * Initialize the application
 */
async function initApp() {
    // Set up event listeners
    setupEventListeners();

    // Load cities for dropdowns
    await loadCities();

    // Check authentication status
    await checkAuthState();

    // Create default profile image in static/uploads
    createDefaultProfileImageIfNeeded();
}

/**
 * Create default profile image if it doesn't exist
 */
function createDefaultProfileImageIfNeeded() {
    // This function is just a placeholder
    // In a real Flask app, we'd check if the default profile image exists and create it if not
    console.log('Default profile image checked');
}

/**
 * Load cities from the API
 */
async function loadCities() {
    try {
        const response = await fetch('/api/get-cities', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });

        const data = await response.json();

        if (data.status === 'success') {
            populateCityDropdowns(data.cities);
        } else {
            console.error('Failed to load cities:', data.message);
        }
```

```
    } catch (error) {
        console.error('Error loading cities:', error);
    }
}

/**
 * Populate all city dropdowns with cities
 * @param {Array} cities - Array of city objects
 */
function populateCityDropdowns(cities) {
    const dropdowns = document.querySelectorAll('#location, #update-city, #register-city');

    dropdowns.forEach(dropdown => {
        // Clear existing options (except the first one)
        const firstOption = dropdown.querySelector('option:first-child');
        dropdown.innerHTML = '';
        dropdown.appendChild(firstOption);

        // Add city options
        cities.forEach(city => {
            const option = document.createElement('option');
            option.value = city.city_name;
            option.textContent = city.city_name;
            dropdown.appendChild(option);
        });
    });
}

/**
 * Check authentication state
 */
async function checkAuthState() {
    try {
        const response = await fetch('/api/auth-status', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });

        const data = await response.json();

        if (data.status === 'success' && data.data.authenticated) {
            updateUIForLoggedInUser();

            if (data.data.is_admin || data.data.is_owner) {
                document.getElementById('admin').classList.remove('hidden');
                await loadAdminData();
            }

            if (data.data.is_owner) {
                document.getElementById('owner').classList.remove('hidden');
            }

            await loadUserProfile();
            await loadUserDonations();
        } else {
```

```
            updateUIForLoggedOutUser();
        }
    } catch (error) {
        console.error('Error checking auth state:', error);
    }
}

/**
 * Update UI for logged in user
 */
function updateUIForLoggedInUser() {
    // Show profile link and logout button
    document.querySelectorAll('.profile-link').forEach(el => el.classList.remove('hidden'));
    document.querySelectorAll('.logout-btn').forEach(el => el.classList.remove('hidden'));

    // Hide login/register buttons
    document.querySelectorAll('.login-btn').forEach(el => el.classList.add('hidden'));
    document.querySelectorAll('.register-btn').forEach(el => el.classList.add('hidden'));

    // Show donation form, hide auth message
    document.querySelector('#donation-form').classList.remove('hidden');
    document.querySelector('.auth-required-message').classList.add('hidden');
}

/**
 * Update UI for logged out user
 */
function updateUIForLoggedOutUser() {
    // Hide profile link and logout button
    document.querySelectorAll('.profile-link').forEach(el => el.classList.add('hidden'));
    document.querySelectorAll('.logout-btn').forEach(el => el.classList.add('hidden'));

    // Show login/register buttons
    document.querySelectorAll('.login-btn').forEach(el => el.classList.remove('hidden'));
    document.querySelectorAll('.register-btn').forEach(el => el.classList.remove('hidden'));

    // Hide profile section and admin/owner sections
    document.getElementById('profile').classList.add('hidden');
    document.getElementById('admin').classList.add('hidden');
    document.getElementById('owner').classList.add('hidden');

    // Hide donation form, show auth message
    document.querySelector('#donation-form').classList.add('hidden');
    document.querySelector('.auth-required-message').classList.remove('hidden');
}

/**
 * Load user profile
 */
async function loadUserProfile() {
    try {
        const response = await fetch('/api/get-profile', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });
```

```
        const data = await response.json();

        if (data.status === 'success') {
            populateProfileData(data.user);
        } else {
            console.error('Failed to load profile:', data.message);
        }
    } catch (error) {
        console.error('Error loading profile:', error);
    }
}

/**
 * Populate profile data in the UI
 * @param {Object} profile - Profile data
 */
function populateProfileData(profile) {
    document.getElementById('profile-name').textContent = profile.name || 'No name provided';
    document.getElementById('profile-username').textContent = profile.username;
    document.getElementById('profile-email').textContent = profile.email || 'No email provided';
    document.getElementById('profile-contact').textContent = profile.phone_or_email || 'No contact
provided';
    document.getElementById('profile-city').textContent = profile.preferred_city || 'No city selected';

    // Set profile image
    const profileImage = document.getElementById('profile-image');
    if (profile.profile_image) {
        profileImage.src = `/uploads/${profile.profile_image}`;
    } else {
        profileImage.src = '/static/uploads/default-profile.png';
    }

    // Set update form values
    document.getElementById('update-name').value = profile.name || '';
    document.getElementById('update-email').value = profile.email || '';
    document.getElementById('update-contact').value = profile.phone_or_email || '';

    // Set city dropdown
    const cityDropdown = document.getElementById('update-city');
    if (profile.preferred_city) {
        Array.from(cityDropdown.options).forEach(option => {
            if (option.value === profile.preferred_city) {
                option.selected = true;
            }
        });
    }
}

/**
 * Load user donations
 */
async function loadUserDonations() {
    try {
        const response = await fetch('/api/get-donations', {
            method: 'POST',
            headers: {
```

```
            'Content-Type': 'application/x-www-form-urlencoded',
        }
    });

    const data = await response.json();

    if (data.status === 'success') {
        displayDonationHistory(data.donations);

        // Update donations count in about section
        document.getElementById('donations-count').textContent = data.donations.length;
    } else {
        console.error('Failed to load donations:', data.message);
    }
  } catch (error) {
    console.error('Error loading donations:', error);
  }
}

/**
 * Display user's donation history
 * @param {Array} donations - Array of donation objects
 */
function displayDonationHistory(donations) {
  const container = document.getElementById('donations-container');

  if (donations.length === 0) {
    container.innerHTML = '<p class="text-center">No donations yet. <a href="#donate">Make your first
donation!</a></p>';
    return;
  }

  container.innerHTML = '';

  donations.forEach(donation => {
    const donationCard = document.createElement('div');
    donationCard.className = `donation-card ${donation.status}`;
    donationCard.setAttribute('data-status', donation.status);

    const donationDate = new Date(donation.created_at).toLocaleDateString();
    const pickupDate = donation.pickup_time ? new Date(donation.pickup_time).toLocaleString() : 'Not
specified';

    donationCard.innerHTML = `
        <div class="donation-header">
            <span class="donation-title">${donation.food_type} (${donation.quantity})</span>
            <span class="donation-status
${donation.status}">${capitalizeFirstLetter(donation.status)}</span>
        </div>
        <div class="donation-details">
            <p><strong>Location:</strong> ${donation.location}</p>
            <p><strong>Date:</strong> ${donationDate}</p>
            <p><strong>Preferred Pickup:</strong> ${pickupDate}</p>
            ${donation.notes ? `<p><strong>Notes:</strong> ${donation.notes}</p>` : ''}
        </div>
        ${donation.status === 'pending' ? `
            <div class="donation-actions">
```

```
                <button class="btn btn-sm btn-danger cancel-donation" data-
id="${donation.id}">Cancel</button>
            </div>
        ` : ''}
    `;

    container.appendChild(donationCard);

    // Add event listener to cancel button
    const cancelBtn = donationCard.querySelector('.cancel-donation');
    if (cancelBtn) {
        cancelBtn.addEventListener('click', function() {
            cancelDonation(donation.id);
        });
    }
  });
}

/**
 * Load admin data
 */
async function loadAdminData() {
  await loadAllDonations();
  await loadAdminCities();

  // If owner, also load users and stats
  if (document.getElementById('owner').classList.contains('hidden') === false) {
    await loadAllUsers();
    await loadAdminStats();
  }
}

/**
 * Load all donations for admin
 */
async function loadAllDonations() {
  try {
    const response = await fetch('/api/get-all-donations', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/x-www-form-urlencoded',
        }
    });

    const data = await response.json();

    if (data.status === 'success') {
        displayAdminDonations(data.donations, 'admin-donations-list');

        // If owner section is visible, also display in owner donations list
        if (document.getElementById('owner').classList.contains('hidden') === false) {
            displayAdminDonations(data.donations, 'owner-donations-list');
        }
    } else {
        console.error('Failed to load admin donations:', data.message);
    }
  } catch (error) {
```

```
        console.error('Error loading admin donations:', error);
    }
}

/**
 * Load all users for owner
 */
async function loadAllUsers() {
    try {
        const response = await fetch('/api/get-users', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });

        const data = await response.json();

        if (data.status === 'success') {
            displayAdminUsers(data.users);
        } else {
            console.error('Failed to load users:', data.message);
        }
    } catch (error) {
        console.error('Error loading users:', error);
    }
}

/**
 * Load cities for admin
 */
async function loadAdminCities() {
    try {
        const response = await fetch('/api/get-cities', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });

        const data = await response.json();

        if (data.status === 'success') {
            displayAdminCities(data.cities);
        } else {
            console.error('Failed to load admin cities:', data.message);
        }
    } catch (error) {
        console.error('Error loading admin cities:', error);
    }
}

/**
 * Load admin stats
 */
async function loadAdminStats() {
    try {
```

```
        const response = await fetch('/api/get-admin-stats', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });

        const data = await response.json();

        if (data.status === 'success') {
            const stats = data.stats;

            // Update user stats
            document.getElementById('stat-total-users').textContent = stats.user_stats.total;
            document.getElementById('stat-admins').textContent = stats.user_stats.admins;
            document.getElementById('stat-regular-users').textContent = stats.user_stats.regular;

            // Update donation stats
            document.getElementById('stat-total-donations').textContent = stats.donation_stats.total;
            document.getElementById('stat-pending-donations').textContent = stats.donation_stats.pending;
            document.getElementById('stat-completed-donations').textContent =
stats.donation_stats.completed;
            document.getElementById('stat-cancelled-donations').textContent = stats.donation_stats.cancelled;

            // Update city stats
            document.getElementById('stat-most-active-city').textContent = stats.city_stats.most_active;
            document.getElementById('stat-total-cities').textContent = stats.city_stats.total;

            // Update activity log
            const activityLog = document.getElementById('activity-log');
            activityLog.innerHTML = '';

            if (stats.recent_activity.length === 0) {
                activityLog.innerHTML = '<p>No recent activity.</p>';
            } else {
                stats.recent_activity.forEach(activity => {
                    const activityItem = document.createElement('div');
                    activityItem.className = 'activity-item';

                    const date = new Date(activity.created_at).toLocaleString();

                    activityItem.innerHTML = `
                        <div class="activity-header">
                            <span class="activity-user">${activity.username}</span>
                            <span class="activity-time">${date}</span>
                        </div>
                        <div class="activity-action">${formatActivityAction(activity.action)}</div>
                        ${activity.details ? `<div class="activity-details">${activity.details}</div>` : ''}
                    `;

                    activityLog.appendChild(activityItem);
                });
            }
        } else {
            console.error('Failed to load admin stats:', data.message);
        }
    } catch (error) {
```

```
        console.error('Error loading admin stats:', error);
    }
}

/**
 * Format activity action for display
 * @param {string} action - Activity action
 * @returns {string} - Formatted action
 */
function formatActivityAction(action) {
    const actionMap = {
        'login': 'Logged in',
        'logout': 'Logged out',
        'profile_update': 'Updated profile',
        'password_update': 'Updated password',
        'profile_image_update': 'Updated profile image',
        'password_reset_request': 'Requested password reset',
        'password_reset_complete': 'Completed password reset',
        'donation_submit': 'Submitted donation',
        'donation_cancel': 'Cancelled donation',
        'donation_status_update': 'Updated donation status',
        'add_city': 'Added city',
        'delete_city': 'Deleted city',
        'grant_admin': 'Granted admin privileges',
        'revoke_admin': 'Revoked admin privileges',
        'delete_user': 'Deleted user'
    };

    return actionMap[action] || action;
}

/**
 * Display donations in admin view
 * @param {Array} donations - Array of donation objects
 * @param {string} containerId - ID of container element
 */
function displayAdminDonations(donations, containerId) {
    const container = document.getElementById(containerId);

    if (donations.length === 0) {
        container.innerHTML = '<p class="text-center">No donations found.</p>';
        return;
    }

    container.innerHTML = '';

    donations.forEach(donation => {
        const donationCard = document.createElement('div');
        donationCard.className = `donation-card ${donation.status}`;
        donationCard.setAttribute('data-status', donation.status);

        const donationDate = new Date(donation.created_at).toLocaleDateString();
        const pickupDate = donation.pickup_time ? new Date(donation.pickup_time).toLocaleString() : 'Not
specified';

        donationCard.innerHTML = `
            <div class="donation-header">
```

```
            <span class="donation-title">${donation.food_type} (${donation.quantity})</span>
            <span class="donation-status
${donation.status}">${capitalizeFirstLetter(donation.status)}</span>
        </div>
        <div class="donation-details">
            <p><strong>User ID:</strong> ${donation.user_id}</p>
            <p><strong>Location:</strong> ${donation.location}</p>
            <p><strong>Date:</strong> ${donationDate}</p>
            <p><strong>Preferred Pickup:</strong> ${pickupDate}</p>
            ${donation.notes ? `<p><strong>Notes:</strong> ${donation.notes}</p>` : ''}
        </div>
        <div class="donation-actions">
            ${donation.status === 'pending' ? `
                <button class="btn btn-sm btn-success complete-donation" data-id="${donation.id}">Mark
Completed</button>
                <button class="btn btn-sm btn-danger cancel-donation" data-
id="${donation.id}">Cancel</button>
            ` : ''}
            ${donation.status === 'cancelled' ? `
                <button class="btn btn-sm btn-warning reopen-donation" data-
id="${donation.id}">Reopen</button>
            ` : ''}
            ${donation.status === 'completed' ? `
                <button class="btn btn-sm btn-warning reopen-donation" data-
id="${donation.id}">Reopen</button>
            ` : ''}
        </div>
    `;

    container.appendChild(donationCard);

    // Add event listeners to buttons
    const completeBtn = donationCard.querySelector('.complete-donation');
    if (completeBtn) {
        completeBtn.addEventListener('click', function() {
            updateDonationStatus(donation.id, 'completed');
        });
    }

    const cancelBtn = donationCard.querySelector('.cancel-donation');
    if (cancelBtn) {
        cancelBtn.addEventListener('click', function() {
            updateDonationStatus(donation.id, 'cancelled');
        });
    }

    const reopenBtn = donationCard.querySelector('.reopen-donation');
    if (reopenBtn) {
        reopenBtn.addEventListener('click', function() {
            updateDonationStatus(donation.id, 'pending');
        });
    }
  });
}

/**
 * Display admin users
```

```
 * @param {Array} users - Array of user objects
 */
function displayAdminUsers(users) {
    const container = document.getElementById('admin-user-list');
    const ownerContainer = document.getElementById('owner-user-list');

    if (users.length === 0) {
        const message = '<p class="text-center">No users found.</p>';
        container.innerHTML = message;
        ownerContainer.innerHTML = message;
        return;
    }

    container.innerHTML = '';
    ownerContainer.innerHTML = '';

    users.forEach(user => {
        const userCard = document.createElement('div');
        let cardClass = 'user-card';
        if (user.is_owner) {
            cardClass += ' owner';
        } else if (user.is_admin) {
            cardClass += ' admin';
        }
        userCard.className = cardClass;

        const joinDate = new Date(user.created_at).toLocaleDateString();

        userCard.innerHTML = `
            <div class="user-info">
                <h4>${user.username}</h4>
                <p><strong>Email:</strong> ${user.email || 'Not provided'}</p>
                <p><strong>Name:</strong> ${user.name || 'Not provided'}</p>
                <p><strong>Joined:</strong> ${joinDate}</p>
                <p><strong>Role:</strong> ${user.is_owner ? 'Owner' : user.is_admin ? 'Admin' : 'User'}</p>
            </div>
            <div class="user-actions">
                ${!user.is_owner ? `
                    ${user.is_admin ? `
                        <button class="btn btn-sm btn-warning toggle-admin" data-id="${user.id}">Remove
Admin</button>
                    ` : `
                        <button class="btn btn-sm btn-info toggle-admin" data-id="${user.id}">Make
Admin</button>
                    `}
                    <button class="btn btn-sm btn-danger delete-user" data-id="${user.id}">Delete</button>
                ` : ''}
            </div>
        `;

        container.appendChild(userCard.cloneNode(true));
        ownerContainer.appendChild(userCard);

        // Add event listeners to owner container buttons
        const toggleAdminBtn = ownerContainer.querySelector(`.toggle-admin[data-id="${user.id}"]`);
        if (toggleAdminBtn) {
            toggleAdminBtn.addEventListener('click', function() {
```

```
            toggleAdminStatus(user.id);
        });
    }

    const deleteUserBtn = ownerContainer.querySelector(`.delete-user[data-id="${user.id}"]`);
    if (deleteUserBtn) {
        deleteUserBtn.addEventListener('click', function() {
            if (confirm(`Are you sure you want to delete user ${user.username}? This cannot be undone.`)) {
                deleteUser(user.id);
            }
        });
    }
  });
}

/**
 * Display admin cities
 * @param {Array} cities - Array of city objects
 */
function displayAdminCities(cities) {
  const container = document.getElementById('admin-city-list');

  if (cities.length === 0) {
    container.innerHTML = '<p class="text-center">No cities found.</p>';
    return;
  }

  container.innerHTML = '';

  cities.forEach(city => {
    const cityCard = document.createElement('div');
    cityCard.className = 'city-card';

    cityCard.innerHTML = `
        <div class="city-info">
            <h4>${city.city_name}</h4>
        </div>
        <div class="city-actions">
            <button class="btn btn-sm btn-danger delete-city" data-id="${city.id}">Delete</button>
        </div>
    `;

    container.appendChild(cityCard);

    // Add event listener to delete button
    const deleteBtn = cityCard.querySelector('.delete-city');
    deleteBtn.addEventListener('click', function() {
        if (confirm(`Are you sure you want to delete city ${city.city_name}?`)) {
            deleteCity(city.id);
        }
    });
  });
}

/**
 * Set up event listeners for various UI elements
 */
```

```
function setupEventListeners() {
    // Login modal
    document.querySelectorAll('.login-btn').forEach(btn => {
        btn.addEventListener('click', function(e) {
            e.preventDefault();
            document.getElementById('login-modal').style.display = 'block';
        });
    });

    // Register modal
    document.querySelectorAll('.register-btn').forEach(btn => {
        btn.addEventListener('click', function(e) {
            e.preventDefault();
            document.getElementById('register-modal').style.display = 'block';
        });
    });

    // Register link in login modal
    document.getElementById('register-link').addEventListener('click', function(e) {
        e.preventDefault();
        document.getElementById('login-modal').style.display = 'none';
        document.getElementById('register-modal').style.display = 'block';
    });

    // Login link in register modal
    document.getElementById('login-link').addEventListener('click', function(e) {
        e.preventDefault();
        document.getElementById('register-modal').style.display = 'none';
        document.getElementById('login-modal').style.display = 'block';
    });

    // Forgot password link
    document.getElementById('forgot-password-link').addEventListener('click', function(e) {
        e.preventDefault();
        document.getElementById('login-modal').style.display = 'none';
        document.getElementById('forgot-password-modal').style.display = 'block';
    });

    // Back to login from forgot password
    document.getElementById('back-to-login').addEventListener('click', function(e) {
        e.preventDefault();
        document.getElementById('forgot-password-modal').style.display = 'none';
        document.getElementById('login-modal').style.display = 'block';
    });

    // Close modals
    document.querySelectorAll('.close-btn').forEach(btn => {
        btn.addEventListener('click', function() {
            this.closest('.modal').style.display = 'none';
        });
    });

    // Profile link
    document.querySelectorAll('.profile-link').forEach(link => {
        link.addEventListener('click', function(e) {
            e.preventDefault();
            document.getElementById('profile').classList.remove('hidden');
```

```
        // Hide other sections
        const sections = document.querySelectorAll('main > section');
        sections.forEach(section => {
            if (section.id !== 'profile') {
                section.classList.add('hidden');
            }
        });

        // Update navigation
        document.querySelectorAll('nav ul li a').forEach(navLink => {
            navLink.classList.remove('active');
        });
        this.classList.add('active');
    });
});

// Section navigation
document.querySelectorAll('nav ul li a:not(.login-btn):not(.logout-btn):not(.profile-link)').forEach(link
=> {
    link.addEventListener('click', function(e) {
        e.preventDefault();

        // Get target section
        const targetId = this.getAttribute('href').substring(1);
        const targetSection = document.getElementById(targetId);

        if (targetSection) {
            // Hide all sections
            const sections = document.querySelectorAll('main > section');
            sections.forEach(section => {
                section.classList.add('hidden');
            });

            // Show target section
            targetSection.classList.remove('hidden');

            // Update navigation
            document.querySelectorAll('nav ul li a').forEach(navLink => {
                navLink.classList.remove('active');
            });
            this.classList.add('active');
        }
    });
});

// Tab navigation
document.querySelectorAll('.tab-btn').forEach(tab => {
    tab.addEventListener('click', function() {
        // Get parent container and target tab content
        const tabsContainer = this.closest('div');
        const tabContentClass = this.getAttribute('data-tab');
        const tabContent = document.querySelector(`.tab-content.${tabContentClass}`);

        if (tabContent) {
            // Remove active class from all tabs and tab contents
            tabsContainer.querySelectorAll('.tab-btn').forEach(t => t.classList.remove('active'));
```

```
            const parent = tabContent.parentElement;
            parent.querySelectorAll('.tab-content').forEach(c => c.classList.remove('active'));

            // Add active class to clicked tab and target tab content
            this.classList.add('active');
            tabContent.classList.add('active');
        }
    });
});

// Form submissions
document.getElementById('login-form').addEventListener('submit', handleLogin);
document.getElementById('register-form').addEventListener('submit', handleRegistration);
document.getElementById('forgot-password-form').addEventListener('submit', handleForgotPassword);
document.querySelectorAll('.logout-btn').forEach(btn => {
    btn.addEventListener('click', handleLogout);
});
document.getElementById('donation-form').addEventListener('submit', handleDonationSubmission);
document.getElementById('profile-update-form').addEventListener('submit', handleProfileUpdate);
document.getElementById('password-update-form').addEventListener('submit', handlePasswordUpdate);
document.getElementById('city-add-form').addEventListener('submit', handleAddCity);

// Profile image upload
document.getElementById('profile-image-upload').addEventListener('change', function() {
    if (this.files && this.files[0]) {
        handleProfileImageUpload(this.files[0]);
    }
});

// Donation filters
document.querySelectorAll('.donation-filters .filter-btn').forEach(btn => {
    btn.addEventListener('click', function() {
        const filter = this.getAttribute('data-filter');

        // Update active button
        this.parentElement.querySelectorAll('.filter-btn').forEach(b => b.classList.remove('active'));
        this.classList.add('active');

        // Filter donations
        const donationCards = document.querySelectorAll('#donations-container .donation-card');
        donationCards.forEach(card => {
            if (filter === 'all' || card.getAttribute('data-status') === filter) {
                card.style.display = 'block';
            } else {
                card.style.display = 'none';
            }
        });
    });
});

// Admin donation filters
document.querySelectorAll('.admin-filters .filter-btn').forEach(btn => {
    btn.addEventListener('click', function() {
        const filter = this.getAttribute('data-filter');
        const container = this.closest('section').querySelector('.admin-donations-list, .owner-donations-list');

        // Update active button
```

```
            this.parentElement.querySelectorAll('.filter-btn').forEach(b => b.classList.remove('active'));
            this.classList.add('active');

            // Filter donations
            const donationCards = container.querySelectorAll('.donation-card');
            donationCards.forEach(card => {
                if (filter === 'all' || card.getAttribute('data-status') === filter) {
                    card.style.display = 'block';
                } else {
                    card.style.display = 'none';
                }
            });
        });
    });

    // Register password strength meter
    document.getElementById('register-password').addEventListener('input', function() {
        updateRegisterPasswordStrengthMeter(this.value);
    });

    // Password update strength meter
    document.getElementById('new-password').addEventListener('input', function() {
        updatePasswordStrengthMeter(this.value);
    });

    // Confirm password validation
    document.getElementById('register-confirm-password').addEventListener('input', function() {
        validateConfirmPassword('register-password', 'register-confirm-password');
    });

    document.getElementById('confirm-password').addEventListener('input', function() {
        validateConfirmPassword('new-password', 'confirm-password');
    });

    // Donate button in hero section
    document.querySelector('.hero-buttons .btn-primary').addEventListener('click', scrollToDonate);
}

/**
 * Handle login form submission
 * @param {Event} e - Form submission event
 */
async function handleLogin(e) {
    e.preventDefault();

    const username = document.getElementById('login-username').value.trim();
    const password = document.getElementById('login-password').value;
    const messageContainer = document.querySelector('#login-form .form-message');

    if (!username || !password) {
        showFormMessage(messageContainer, 'error', 'Please enter both username and password.');
        return;
    }

    try {
        // Show loading state
        const submitButton = document.querySelector('#login-form button[type="submit"]');
```

```
        submitButton.disabled = true;
        submitButton.textContent = 'Logging in...';

        const formData = new URLSearchParams();
        formData.append('username', username);
        formData.append('password', password);

        const response = await fetch('/api/login', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            // Hide modal
            document.getElementById('login-modal').style.display = 'none';

            // Update UI
            updateUIForLoggedInUser();

            // Check admin/owner status
            if (data.user.is_admin || data.user.is_owner) {
                document.getElementById('admin').classList.remove('hidden');
                await loadAdminData();
            }

            if (data.user.is_owner) {
                document.getElementById('owner').classList.remove('hidden');
            }

            // Load user data
            await loadUserProfile();
            await loadUserDonations();

            // Reset form
            document.getElementById('login-form').reset();
            messageContainer.classList.remove('error', 'success');
            messageContainer.style.display = 'none';
        } else {
            showFormMessage(messageContainer, 'error', data.message || 'Login failed. Please try again.');
        }
    } catch (error) {
        console.error('Error during login:', error);
        showFormMessage(messageContainer, 'error', 'An error occurred during login. Please try again.');
    } finally {
        // Reset button
        const submitButton = document.querySelector('#login-form button[type="submit"]');
        submitButton.disabled = false;
        submitButton.textContent = 'Login';
    }
}

/**
```

```
 * Handle registration form submission
 * @param {Event} e - Form submission event
 */
async function handleRegistration(e) {
    e.preventDefault();

    const username = document.getElementById('register-username').value.trim();
    const password = document.getElementById('register-password').value;
    const confirmPassword = document.getElementById('register-confirm-password').value;
    const email = document.getElementById('register-email').value.trim();
    const name = document.getElementById('register-name').value.trim();
    const contact = document.getElementById('register-contact').value.trim();
    const city = document.getElementById('register-city').value;
    const messageContainer = document.querySelector('#register-form .form-message');

    if (!username || !password || !confirmPassword) {
        showFormMessage(messageContainer, 'error', 'Please fill in all required fields.');
        return;
    }

    if (password !== confirmPassword) {
        showFormMessage(messageContainer, 'error', 'Passwords do not match.');
        return;
    }

    if (password.length < 8) {
        showFormMessage(messageContainer, 'error', 'Password must be at least 8 characters long.');
        return;
    }

    try {
        // Show loading state
        const submitButton = document.querySelector('#register-form button[type="submit"]');
        submitButton.disabled = true;
        submitButton.textContent = 'Registering...';

        const formData = new URLSearchParams();
        formData.append('username', username);
        formData.append('password', password);
        if (email) formData.append('email', email);
        if (name) formData.append('name', name);
        if (contact) formData.append('contact', contact);
        if (city) formData.append('city', city);

        const response = await fetch('/api/register', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            showFormMessage(messageContainer, 'success', data.message || 'Registration successful! You can
now login.');
```

```
            // Reset form
            document.getElementById('register-form').reset();

            // Switch to login after 2 seconds
            setTimeout(() => {
                document.getElementById('register-modal').style.display = 'none';
                document.getElementById('login-modal').style.display = 'block';
                messageContainer.classList.remove('error', 'success');
                messageContainer.style.display = 'none';
            }, 2000);
        } else {
            showFormMessage(messageContainer, 'error', data.message || 'Registration failed. Please try
again.');
        }
    } catch (error) {
        console.error('Error during registration:', error);
        showFormMessage(messageContainer, 'error', 'An error occurred during registration. Please try
again.');
    } finally {
        // Reset button
        const submitButton = document.querySelector('#register-form button[type="submit"]');
        submitButton.disabled = false;
        submitButton.textContent = 'Register';
    }
}

/**
 * Handle forgot password form submission
 * @param {Event} e - Form submission event
 */
async function handleForgotPassword(e) {
    e.preventDefault();

    const usernameOrEmail = document.getElementById('forgot-username').value.trim();
    const messageContainer = document.querySelector('#forgot-password-form .form-message');

    if (!usernameOrEmail) {
        showFormMessage(messageContainer, 'error', 'Please enter your username or email.');
        return;
    }

    try {
        // Show loading state
        const submitButton = document.querySelector('#forgot-password-form button[type="submit"]');
        submitButton.disabled = true;
        submitButton.textContent = 'Processing...';

        const formData = new URLSearchParams();
        formData.append('username_or_email', usernameOrEmail);

        const response = await fetch('/api/forgot-password', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
```

```
        });

        const data = await response.json();

        if (data.status === 'success') {
            showFormMessage(messageContainer, 'success', data.message || 'Password reset instructions sent.
Please check your email.');

            // Reset form
            document.getElementById('forgot-password-form').reset();
        } else {
            showFormMessage(messageContainer, 'error', data.message || 'Failed to process request. Please try
again.');
        }
    } catch (error) {
        console.error('Error during forgot password:', error);
        showFormMessage(messageContainer, 'error', 'An error occurred. Please try again.');
    } finally {
        // Reset button
        const submitButton = document.querySelector('#forgot-password-form button[type="submit"]');
        submitButton.disabled = false;
        submitButton.textContent = 'Reset Password';
    }
}

/**
 * Handle logout
 * @param {Event} e - Click event
 */
async function handleLogout(e) {
    e.preventDefault();

    try {
        const response = await fetch('/api/logout', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            }
        });

        const data = await response.json();

        if (data.status === 'success') {
            // Update UI
            updateUIForLoggedOutUser();

            // Redirect to home section
            const homeSection = document.getElementById('home');
            homeSection.classList.remove('hidden');

            // Update navigation
            document.querySelectorAll('nav ul li a').forEach(link => {
                link.classList.remove('active');
                if (link.getAttribute('href') === '#home') {
                    link.classList.add('active');
                }
            });
```

```
        } else {
            console.error('Logout failed:', data.message);
        }
    } catch (error) {
        console.error('Error during logout:', error);
    }
}

/**
 * Handle donation form submission
 * @param {Event} e - Form submission event
 */
async function handleDonationSubmission(e) {
    e.preventDefault();

    const foodType = document.getElementById('food-type').value;
    const quantity = document.getElementById('quantity').value.trim();
    const location = document.getElementById('location').value;
    const pickupTime = document.getElementById('pickup-date').value;
    const notes = document.getElementById('notes').value.trim();

    if (!foodType || !quantity || !location) {
        alert('Please fill in all required fields.');
        return;
    }

    try {
        // Show loading state
        const submitButton = document.querySelector('#donation-form button[type="submit"]');
        submitButton.disabled = true;
        submitButton.textContent = 'Submitting...';

        const formData = new URLSearchParams();
        formData.append('food_type', foodType);
        formData.append('quantity', quantity);
        formData.append('location', location);
        if (pickupTime) formData.append('pickup_time', pickupTime);
        if (notes) formData.append('notes', notes);

        const response = await fetch('/api/submit-donation', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert('Donation submitted successfully!');

            // Reset form
            document.getElementById('donation-form').reset();

            // Reload user donations
            await loadUserDonations();
```

```
            // Reload admin donations if admin/owner
            if (!document.getElementById('admin').classList.contains('hidden')) {
                await loadAllDonations();
            }
        } else {
            alert(data.message || 'Failed to submit donation. Please try again.');
        }
    } catch (error) {
        console.error('Error submitting donation:', error);
        alert('An error occurred while submitting your donation. Please try again.');
    } finally {
        // Reset button
        const submitButton = document.querySelector('#donation-form button[type="submit"]');
        submitButton.disabled = false;
        submitButton.textContent = 'Submit Donation';
    }
}

/**
 * Handle profile update form submission
 * @param {Event} e - Form submission event
 */
async function handleProfileUpdate(e) {
    e.preventDefault();

    const name = document.getElementById('update-name').value.trim();
    const email = document.getElementById('update-email').value.trim();
    const contact = document.getElementById('update-contact').value.trim();
    const city = document.getElementById('update-city').value;

    try {
        // Show loading state
        const submitButton = document.querySelector('#profile-update-form button[type="submit"]');
        submitButton.disabled = true;
        submitButton.textContent = 'Updating...';

        const formData = new URLSearchParams();
        if (name) formData.append('name', name);
        if (email) formData.append('email', email);
        if (contact) formData.append('contact', contact);
        if (city) formData.append('city', city);

        const response = await fetch('/api/update-profile', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert('Profile updated successfully!');

            // Reload user profile
```

```
            await loadUserProfile();
        } else {
            alert(data.message || 'Failed to update profile. Please try again.');
        }
    } catch (error) {
        console.error('Error updating profile:', error);
        alert('An error occurred while updating your profile. Please try again.');
    } finally {
        // Reset button
        const submitButton = document.querySelector('#profile-update-form button[type="submit"]');
        submitButton.disabled = false;
        submitButton.textContent = 'Update Profile';
    }
}

/**
 * Handle password update form submission
 * @param {Event} e - Form submission event
 */
async function handlePasswordUpdate(e) {
    e.preventDefault();

    const currentPassword = document.getElementById('current-password').value;
    const newPassword = document.getElementById('new-password').value;
    const confirmPassword = document.getElementById('confirm-password').value;

    if (!currentPassword || !newPassword || !confirmPassword) {
        alert('Please fill in all fields.');
        return;
    }

    if (newPassword !== confirmPassword) {
        alert('New password and confirm password do not match.');
        return;
    }

    if (newPassword.length < 8) {
        alert('New password must be at least 8 characters long.');
        return;
    }

    try {
        // Show loading state
        const submitButton = document.querySelector('#password-update-form button[type="submit"]');
        submitButton.disabled = true;
        submitButton.textContent = 'Updating...';

        const formData = new URLSearchParams();
        formData.append('current_password', currentPassword);
        formData.append('new_password', newPassword);

        const response = await fetch('/api/update-password', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
```

```
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert('Password updated successfully!');

            // Reset form
            document.getElementById('password-update-form').reset();
        } else {
            alert(data.message || 'Failed to update password. Please try again.');
        }
    } catch (error) {
        console.error('Error updating password:', error);
        alert('An error occurred while updating your password. Please try again.');
    } finally {
        // Reset button
        const submitButton = document.querySelector('#password-update-form button[type="submit"]');
        submitButton.disabled = false;
        submitButton.textContent = 'Update Password';
    }
}

/**
 * Handle profile image upload
 * @param {File} file - Image file to upload
 */
async function handleProfileImageUpload(file) {
    if (!file) {
        return;
    }

    // Check file type
    const fileType = file.type;
    if (!['image/jpeg', 'image/png', 'image/gif'].includes(fileType)) {
        alert('Only JPEG, PNG, and GIF images are allowed.');
        return;
    }

    // Check file size (max 2MB)
    if (file.size > 2 * 1024 * 1024) {
        alert('Image size exceeds 2MB limit.');
        return;
    }

    try {
        // Create form data
        const formData = new FormData();
        formData.append('image', file);

        const response = await fetch('/api/update-profile-image', {
            method: 'POST',
            body: formData
        });

        const data = await response.json();
```

```
        if (data.status === 'success') {
            // Update profile image
            document.getElementById('profile-image').src = data.image_url;
            alert('Profile image updated successfully!');
        } else {
            alert(data.message || 'Failed to update profile image. Please try again.');
        }
    } catch (error) {
        console.error('Error uploading profile image:', error);
        alert('An error occurred while uploading your profile image. Please try again.');
    }
}

/**
 * Cancel a donation
 * @param {number} donationId - ID of donation to cancel
 */
async function cancelDonation(donationId) {
    if (!confirm('Are you sure you want to cancel this donation?')) {
        return;
    }

    try {
        const formData = new URLSearchParams();
        formData.append('donation_id', donationId);

        const response = await fetch('/api/cancel-donation', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert('Donation cancelled successfully!');

            // Reload donations
            await loadUserDonations();

            // Reload admin donations if admin/owner
            if (!document.getElementById('admin').classList.contains('hidden')) {
                await loadAllDonations();
            }
        } else {
            alert(data.message || 'Failed to cancel donation. Please try again.');
        }
    } catch (error) {
        console.error('Error cancelling donation:', error);
        alert('An error occurred while cancelling your donation. Please try again.');
    }
}

/**
 * Update donation status (admin/owner only)
```

```
 * @param {number} donationId - ID of donation to update
 * @param {string} status - New status ('pending', 'completed', or 'cancelled')
 */
async function updateDonationStatus(donationId, status) {
    try {
        const formData = new URLSearchParams();
        formData.append('donation_id', donationId);
        formData.append('status', status);

        const response = await fetch('/api/update-donation-status', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert(`Donation status updated to ${status}.`);

            // Reload admin donations
            await loadAllDonations();
        } else {
            alert(data.message || 'Failed to update donation status. Please try again.');
        }
    } catch (error) {
        console.error('Error updating donation status:', error);
        alert('An error occurred while updating donation status. Please try again.');
    }
}

/**
 * Handle city add form submission
 * @param {Event} e - Form submission event
 */
async function handleAddCity(e) {
    e.preventDefault();

    const cityName = document.getElementById('city-name').value.trim();

    if (!cityName) {
        alert('Please enter a city name.');
        return;
    }

    try {
        // Show loading state
        const submitButton = document.querySelector('#city-add-form button[type="submit"]');
        submitButton.disabled = true;
        submitButton.textContent = 'Adding...';

        const formData = new URLSearchParams();
        formData.append('city_name', cityName);

        const response = await fetch('/api/add-city', {
```

```
        method: 'POST',
        headers: {
            'Content-Type': 'application/x-www-form-urlencoded',
        },
        body: formData.toString()
    });

    const data = await response.json();

    if (data.status === 'success') {
        alert('City added successfully!');

        // Reset form
        document.getElementById('city-add-form').reset();

        // Reload cities
        await loadCities();
        await loadAdminCities();
    } else {
        alert(data.message || 'Failed to add city. Please try again.');
    }
  } catch (error) {
    console.error('Error adding city:', error);
    alert('An error occurred while adding the city. Please try again.');
  } finally {
    // Reset button
    const submitButton = document.querySelector('#city-add-form button[type="submit"]');
    submitButton.disabled = false;
    submitButton.textContent = 'Add City';
  }
}

/**
 * Delete a city (admin/owner only)
 * @param {number} cityId - ID of city to delete
 */
async function deleteCity(cityId) {
  try {
    const formData = new URLSearchParams();
    formData.append('city_id', cityId);

    const response = await fetch('/api/delete-city', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/x-www-form-urlencoded',
        },
        body: formData.toString()
    });

    const data = await response.json();

    if (data.status === 'success') {
        alert('City deleted successfully!');

        // Reload cities
        await loadCities();
        await loadAdminCities();
```

```
        } else {
            alert(data.message || 'Failed to delete city. Please try again.');
        }
    } catch (error) {
        console.error('Error deleting city:', error);
        alert('An error occurred while deleting the city. Please try again.');
    }
}

/**
 * Toggle admin status for a user (owner only)
 * @param {number} userId - ID of user to toggle admin status
 */
async function toggleAdminStatus(userId) {
    try {
        const formData = new URLSearchParams();
        formData.append('user_id', userId);

        const response = await fetch('/api/toggle-admin', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert(data.message || 'Admin status updated successfully!');

            // Reload users
            await loadAllUsers();
        } else {
            alert(data.message || 'Failed to update admin status. Please try again.');
        }
    } catch (error) {
        console.error('Error toggling admin status:', error);
        alert('An error occurred while updating admin status. Please try again.');
    }
}

/**
 * Delete a user (owner only)
 * @param {number} userId - ID of user to delete
 */
async function deleteUser(userId) {
    try {
        const formData = new URLSearchParams();
        formData.append('user_id', userId);

        const response = await fetch('/api/delete-user', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded',
            },
            body: formData.toString()
```

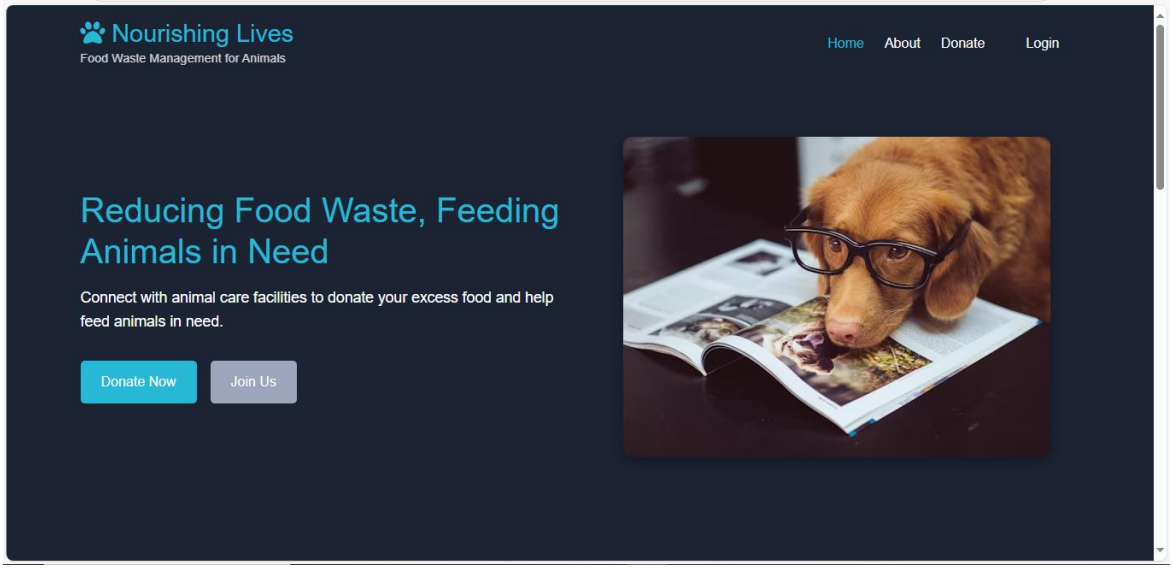
```
        });

        const data = await response.json();

        if (data.status === 'success') {
            alert(data.message || 'User deleted successfully!');

            // Reload users
            await loadAllUsers();

            // Reload stats if we're in the owner dashboard
            if (!document.getElementById('owner').classList.contains('hidden')) {
                await loadAdminStats();
            }
        } else {
            alert(data.message || 'Failed to delete user. Please try again.');
        }
    } catch (error) {
        console.error('Error deleting user:', error);
        alert('An error occurred while deleting the user. Please try again.');
    }
}

/**
 * Update password strength meter for the password update form
 * @param {string} password - Password to check
 */
function updatePasswordStrengthMeter(password) {
    const strength = getPasswordStrength(password);
    const meter = document.querySelector('#password-update-form .strength-meter');
    const text = document.querySelector('#password-update-form .strength-text');

    // Remove all classes
    meter.classList.remove('weak', 'medium', 'strong');

    if (password.length > 0) {
        meter.classList.add(strength);
        text.textContent = `Password strength: ${capitalizeFirstLetter(strength)}`;
    } else {
        text.textContent = 'Password strength';
    }
}

/**
 * Update password strength meter for the registration form
 * @param {string} password - Password to check
 */
function updateRegisterPasswordStrengthMeter(password) {
    const strength = getPasswordStrength(password);
    const meter = document.querySelector('#register-form .strength-meter');
    const text = document.querySelector('#register-form .strength-text');

    // Remove all classes
    meter.classList.remove('weak', 'medium', 'strong');

    if (password.length > 0) {
        meter.classList.add(strength);
```

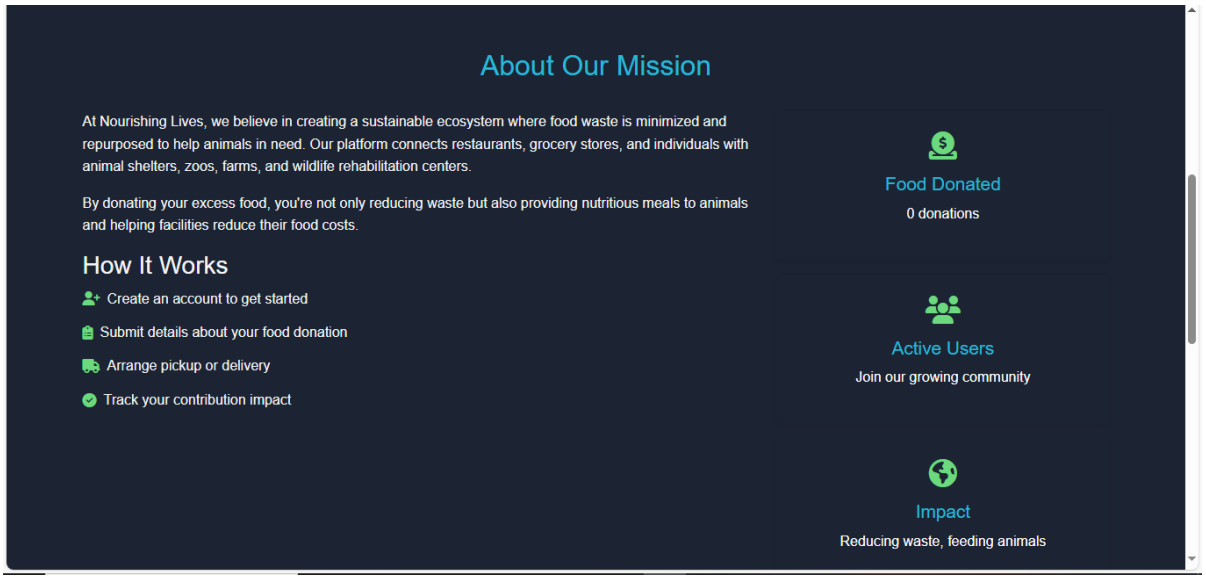
```
        text.textContent = `Password strength: ${capitalizeFirstLetter(strength)}`;
    } else {
        text.textContent = 'Password strength';
    }
}

/**
 * Get password strength
 * @param {string} password - Password to check
 * @returns {string} - 'weak', 'medium', or 'strong'
 */
function getPasswordStrength(password) {
    if (password.length < 8) {
        return 'weak';
    }

    const hasLowerCase = /[a-z]/.test(password);
    const hasUpperCase = /[A-Z]/.test(password);
    const hasNumbers = /\d/.test(password);
    const hasSpecialChars = /[!@#$%^&*(),.?":{}|<>]/.test(password);

    const score = [hasLowerCase, hasUpperCase, hasNumbers, hasSpecialChars].filter(Boolean).length;

    if (score <= 2) {
        return 'weak';
    } else if (score === 3) {
        return 'medium';
    } else {
        return 'strong';
    }
}

/**
 * Validate confirm password field
 * @param {string} passwordId - ID of password field
 * @param {string} confirmId - ID of confirm password field
 */
function validateConfirmPassword(passwordId, confirmId) {
    const password = document.getElementById(passwordId).value;
    const confirmPassword = document.getElementById(confirmId).value;
    const confirmField = document.getElementById(confirmId);

    if (confirmPassword.length > 0) {
        if (password === confirmPassword) {
            confirmField.setCustomValidity('');
        } else {
            confirmField.setCustomValidity('Passwords do not match');
        }
    } else {
        confirmField.setCustomValidity('');
    }
}

/**
 * Show form message
 * @param {Element} container - Form message container
 * @param {string} type - Message type ('error', 'success', or 'info')
```

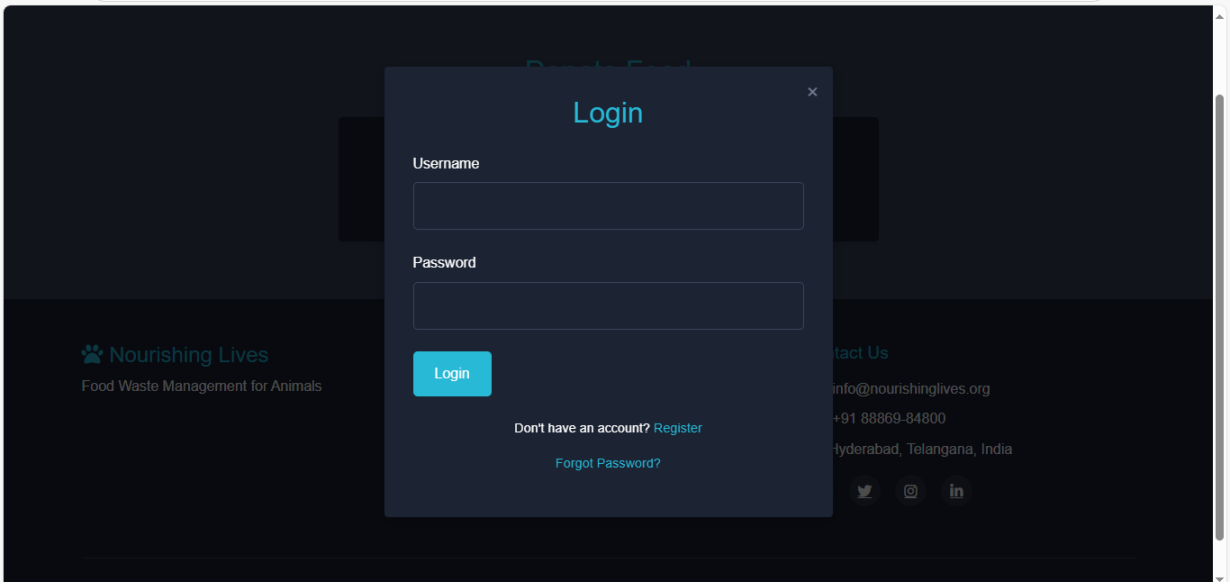
```
 * @param {string} message - Message to display
 */
function showFormMessage(container, type, message) {
    container.textContent = message;
    container.classList.remove('error', 'success', 'info');
    container.classList.add(type);
    container.style.display = 'block';
}

/**
 * Scroll to donate section
 */
function scrollToDonate() {
    const donateSection = document.getElementById('donate');

    // Show donate section (and hide others)
    const sections = document.querySelectorAll('main > section');
    sections.forEach(section => {
        if (section.id === 'donate') {
            section.classList.remove('hidden');
        } else {
            section.classList.add('hidden');
        }
    });

    // Update navigation
    document.querySelectorAll('nav ul li a').forEach(link => {
        link.classList.remove('active');
        if (link.getAttribute('href') === '#donate') {
            link.classList.add('active');
        }
    });

    // Scroll to section
    donateSection.scrollIntoView({ behavior: 'smooth' });
}

/**
 * Capitalize first letter of a string
 * @param {string} string - String to capitalize
 * @returns {string} - Capitalized string
 */
function capitalizeFirstLetter(string) {
    return string.charAt(0).toUpperCase() + string.slice(1);
}
```

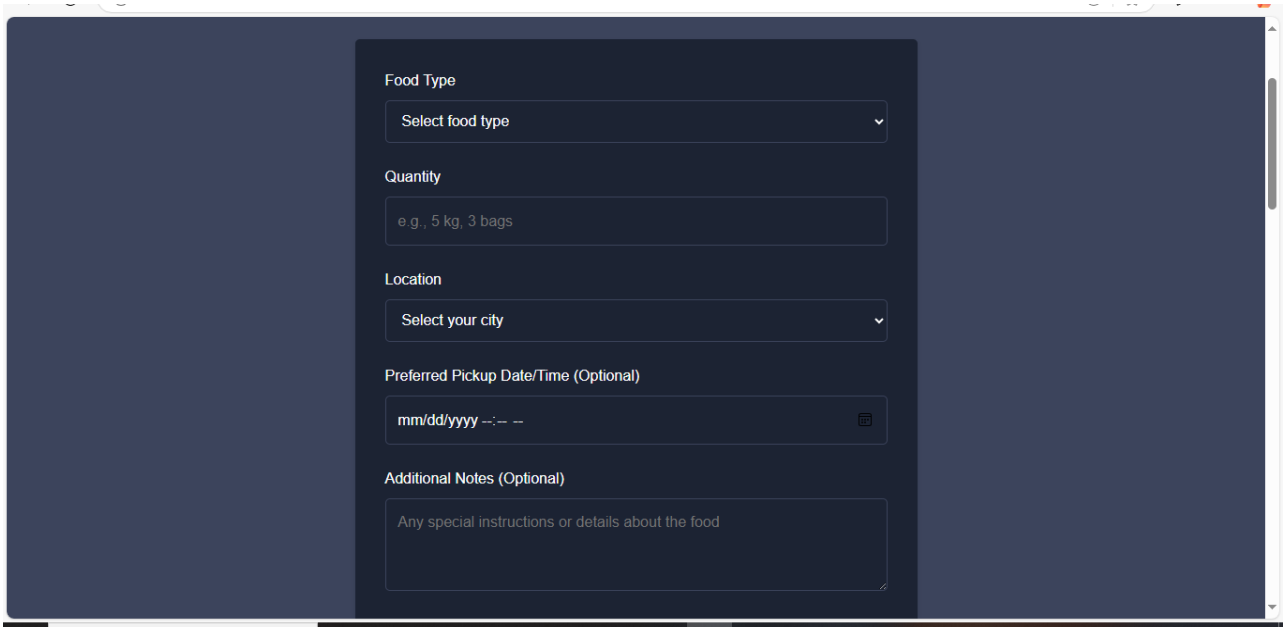
Partial Implementation

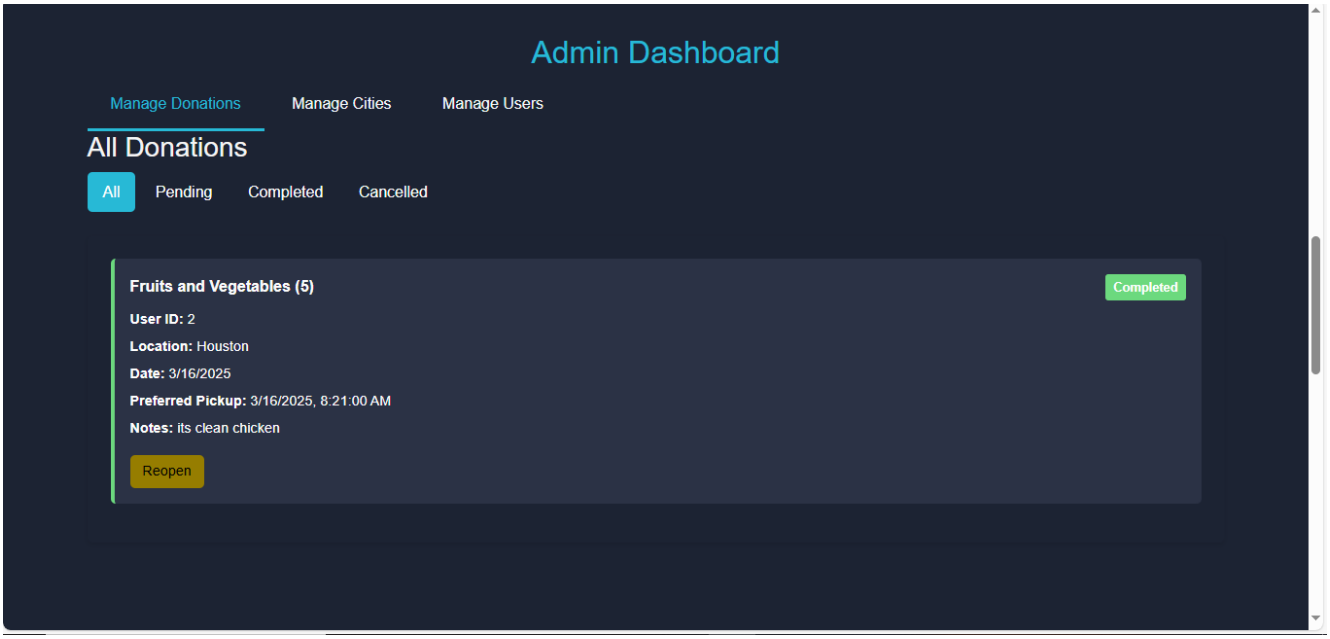
Partial Implementation

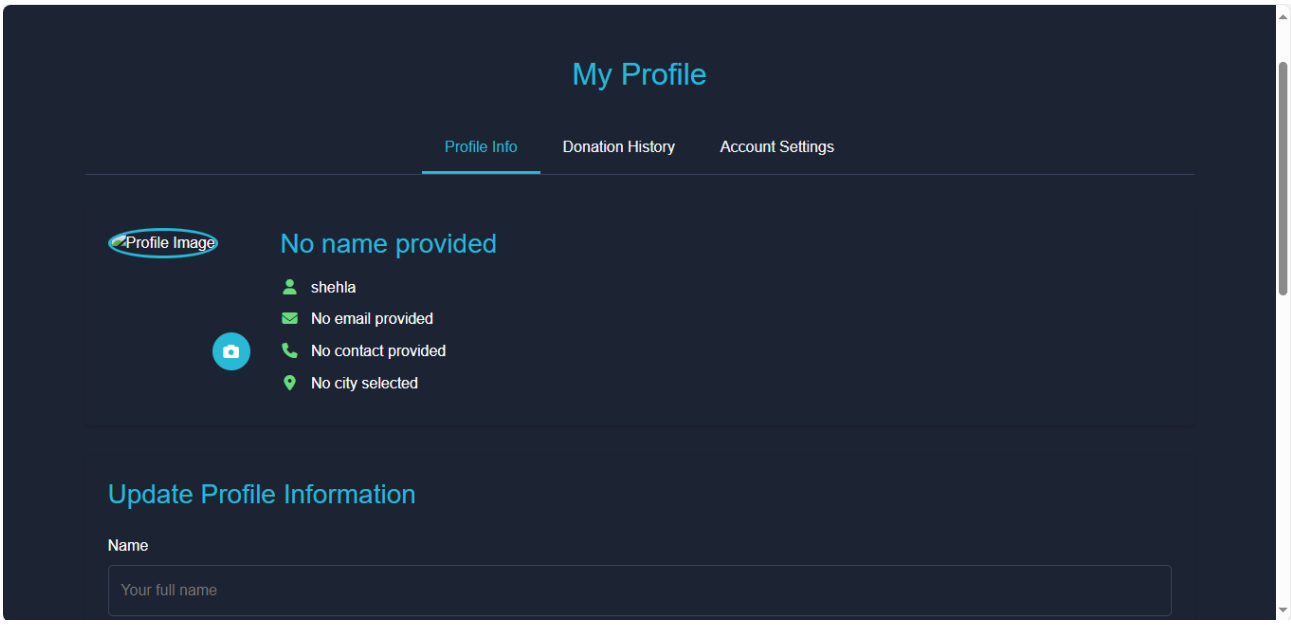





Login
Username
Password
Login
Don't have an account? Register
Forgot Password?
Nourishing Lives
Food Waste Management for Animals
info@nourishinglives.org
+91 88869-84800
Hyderabad, Telangana, India


Food Type
Select food type
Quantity
e.g., 5 kg, 3 bags
Location
Select your city
Preferred Pickup Date/Time (Optional)
mm/dd/yyyy --:-- --
Additional Notes (Optional)
Any special instructions or details about the food


Admin Dashboard
Manage Donations
Manage Cities
Manage Users
All Donations
All
Pending
Completed
Cancelled
Fruits and Vegetables (5)
Completed
User ID: 2
Location: Houston
Date: 3/16/2025
Preferred Pickup: 3/16/2025, 8:21:00 AM
Notes: its clean chicken
Reopen


My Profile
Profile Info
Donation History
Account Settings
Profile Image
No name provided
shehla
No email provided
No contact provided
No city selected
Update Profile Information
Name
Your full name

Conclusion

The Food Waste Management System for Animals is a web-based platform that connects food donors with animal shelters to reduce food waste and support animal welfare. Using a frontend and backend the system allows donors to register, submit food donations and help the animals. It is a website where we made a n attempt to touch each life using the waste at our places.

Future Enhancement:

1. Live Updates Page

Food Pickup & Delivery Status:

Show a live tracking system where donors can see the progress (e.g., "Food Picked Up," "On the Way," "Delivered").

Use Google Maps API to display the location of the delivery in real time.

Real-Time Notifications:

Send email or SMS alerts when food is picked up, in transit, and delivered.

Use push notifications if you plan to add a mobile-friendly version.

2. Live Pictures of Animals Being Fed

Automated Photo Uploads:

Volunteers or staff can take photos of the animals eating and upload them instantly.

Use a gallery or carousel feature on the website to display the latest photos.

Timestamp & Donor Acknowledgment:

Show the date & time of the feeding.

(Optional) Mention the donor’s name (if they agree to it).

Bibliography

Bibliography

- https://openai.com/index/chatgpt/
- https://www.canva.com/
- https://www.geeksforgeeks.org/
- https://github.com/